STUDY GUIDE · SYSTEMS & DATA FOUNDATIONS

Data Engineering & Systems Fundamentals

A Beginner-to-Advanced Study Guide

The four durable pillars that everything — including AI — runs on:
Databases · Distributed Systems · Networking · Concurrency

Prepared 2026-06-21 · Read the sections in order

Table of Contents

How to Use This Guide

Welcome. This guide is written for one person in particular: the **self-taught developer** who can already build things, but who has never been formally taught how the machines underneath actually work. If you have ever shipped a feature that worked on your laptop and then fell over in production, or written a database query that mysteriously took ten seconds, or nodded along when someone said "it's a network issue" without really knowing what they meant, this guide is for you.

We assume **almost nothing**. Every technical term is defined in plain words the first time it appears. We start from the absolute basics and climb gradually. You do not need a computer science degree. You need curiosity and a little patience.

The four pillars and how they connect

Modern systems rest on four foundations, and this guide is built around them:

- **The machine** (Sections 2–3): how a single computer runs your code, and how it does many things at once. Everything else sits on top of this.
- **Storage** (Sections 4–6): how databases keep your data safe, find it fast, and grow when one machine is no longer enough.
- **The network** (Sections 7–8): how data travels between machines, and how the web stack (the layers that make websites work) is assembled.
- **Scale** (Sections 9–11): how many computers cooperate as one system, how data moves and is reshaped in bulk, and finally how to design a real system by weighing trade-offs.

These pillars are not separate topics; they are layers of the same stack. A slow web page might be a thread problem (Section 3), a missing database index (Section 5), or a network round-trip problem (Section 7). By the end, you will be able to reason across all of them.

> **Analogy:** Think of building a house. The machine is the foundation, storage is the rooms where you keep your things, the network is the plumbing and wiring connecting everything, and scale is what you do when one house becomes a whole street. You cannot understand the street without first understanding one house.

How to read it

Read the sections **in order**, at least the first time. Each one builds vocabulary and intuition the next one assumes. Section 6 (scaling databases) leans on ideas from Section 5 (how databases store data); Section 9 (distributed systems) leans on Sections 7–8 (networking). Skipping ahead is the fastest way to feel lost.

Best practice: Read each section twice. The first pass for the big shape, the second pass slowly, pausing at every callout box. The boxes hold the lessons that take engineers years to learn the hard way.

How to get the most out of it

- When you meet a worked example with small numbers, **do the arithmetic yourself**. The numbers are small on purpose so you can.
- After each section, try to explain the core idea out loud as if to a friend. If you cannot, re-read it.
- Connect it to your own work. Next time a query is slow, ask: which layer is this?

What you will be able to do afterwards

You will read a system design discussion and follow it. You will diagnose why something is slow or breaking by reasoning about layers instead of guessing. You will choose between SQL and NoSQL, TCP and UDP, batch and streaming, with reasons. And you will design a modest real-world system and defend your trade-offs. That skill set is **durable**: frameworks come and go, but processes, indexes, TCP, and consensus have lasted decades and will outlast the current hype.

Key takeaway: This guide turns the invisible machinery beneath your code into something you can see, reason about, and design with. Read it in order, do the small examples, and revisit the callouts.

The Big Picture: Why Systems Fundamentals Are Durable

Welcome. Before we touch a single database or network protocol, let's answer the most important question: *why learn this stuff at all, and why will it still be useful in twenty years?* This section is the map for the whole guide. Everything that follows hangs off the ideas here.

Two words to define first: "data engineering" and "systems fundamentals"

Data engineering is the job of *moving, storing, shaping, and serving data reliably and at scale*. "At scale" means it keeps working when the amount of data, or the number of users, gets very large. A data engineer builds the pipes and storage that other people use: the analysts who make charts, the apps customers click on, the machine-learning models that need to be fed. Think of it as plumbing for information.

Systems fundamentals are the layer *underneath* that work: how a computer actually executes a task. How bytes (a byte is 8 bits, the smallest unit of data a computer normally handles) move through the CPU, the memory, the disk, and the network. How separate machines and separate threads of work coordinate without stepping on each other.

The thesis of this entire guide is simple: **you cannot build good data systems without understanding the machine underneath them.** Every design choice — batch or stream? add an index or scan the whole table? copy the data or split it across machines? use a lock or a queue? — is, at bottom, a bet about a *physical resource* and how much that resource costs. If you don't understand the resource, you're guessing.

> **Key takeaway:** Data engineering is the *what* (move and serve data). Systems fundamentals are the *why and how* (what the machine is physically doing). The second is the foundation that makes the first reliable.

The four pillars (and why a database is secretly all four)

Almost everything in computer systems rests on four load-bearing topics. Picture them as four faces of one single problem: *"get the right data to the right place, correctly, fast enough."*

- **Databases — the *storage* pillar.** How data is saved on durable media (storage that survives a power cut), indexed (organized so you can find things fast), queried, and kept correct. This covers things like B-trees and LSM-trees (two ways of organizing data on disk — covered later), transactions, and indexes.
- **Distributed systems — the *scale* + *failure* pillar.** What you do when one machine is not enough: copy data to several machines (replication), split data across machines (partitioning or sharding), get machines to agree (consensus), and survive machines crashing.

- **Networking — the *movement* pillar.** How bytes physically travel between machines: packets, the TCP/IP protocols, the hard speed limit set by the speed of light, and the difference between bandwidth and latency (defined below).
- **Concurrency — the *coordination* pillar.** Doing many things at once on *one* machine without corrupting your data: threads, locks, race conditions, and async (asynchronous) input/output.

Here's the punchline that ties them together: a **distributed database is all four pillars at once.** It is a database (pillar 1), copied over a network (pillar 3), accessed by many users concurrently (pillar 4), spread across many machines (pillar 2). Learn the four pillars and you can reason about the hardest systems out there.

> **Analogy:** Think of one click on a website as a parcel's journey. It travels down a road (networking), is handled by busy workers sorting many parcels at once (concurrency), arrives at a warehouse with branches in several cities (distributed systems), and is finally placed on a physical shelf (the database / storage). One click touches all four.

Why these skills barely change in 30 years

Programming languages, cloud providers, and "framework of the month" tools churn constantly. Yet the gap between the fastest and slowest places a computer can store data has stayed roughly the same for *decades*. Why? Because fundamentals are bounded by **physics and information theory, not by fashion.**

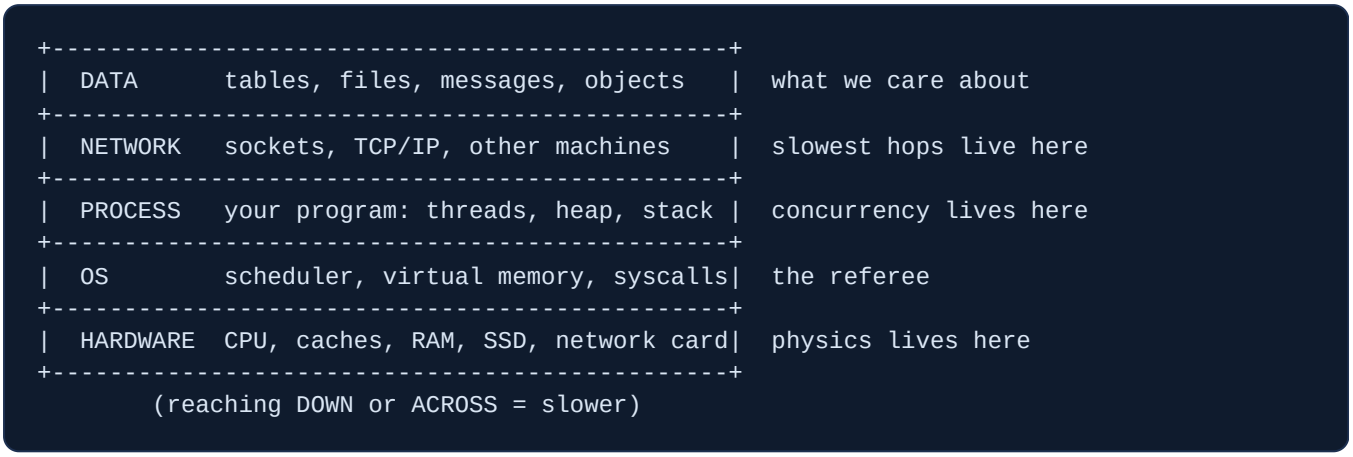
The speed of light is a hard floor. The cost of a "cache miss" (asking for data and finding it isn't in the fast nearby memory, so you must fetch it from somewhere slower) is a hard floor. The impossibility of instantly agreeing across an unreliable network is a hard floor. Nobody can buy their way past these with money or a newer library.

A famous reference table of latency numbers (we'll see it below) was popularized around 2012. In 2026 it is *still essentially correct*. Only disks and networks got faster, and only by a constant factor — not by changing category. So time you spend understanding *why* a cache miss is expensive pays off for your whole career. Time spent memorizing one specific tool's exact function names depreciates within a couple of years.

> **Common mistake:** Believing AI and large language models (LLMs) make fundamentals obsolete. The opposite is true. LLMs are great at the churny surface (boilerplate code, glue between APIs) and *weakest* at deep system reasoning — figuring out *why* something is slow or wrong. An AI will happily write code that is $O(n^2)$ (gets dramatically slower as input grows) or makes a network call inside a loop. Only someone with the mental model catches it. Fundamentals are how you *supervise* AI output instead of being misled by it. And modern machine learning is itself a brutal systems problem — its bottleneck is usually data movement and I/O (input/output: reading and writing data), not the math.

The mental model: a computer as a stack of layers

The cleanest way to picture a computer is as a stack you pass *through* to get work done. Each layer up adds abstraction (hiding messy detail) and, usually, more delay.



Three ideas to lock in:

- **The OS (operating system) is a referee.** It multiplexes scarce hardware — meaning it shares one CPU and one pool of memory (RAM) among many running programs, taking turns so fast it looks simultaneous, while stopping programs from corrupting each other.
- **A syscall (system call) is a controlled trip down to the OS** — for example, "open this file" or "send this on the network." It costs roughly a microsecond or more (a microsecond, written µs, is one millionth of a second). That's cheap compared to disk, but expensive compared to a plain function call inside your program.
- **Crossing a layer boundary costs time.** The further down or across you reach, the slower it gets. Reading a variable in your program touches just the CPU and RAM. Reading a row from a database on another continent touches *every* layer, including the network. **Most performance bugs are simply "we accidentally reached across an expensive boundary inside a loop."**

The latency numbers every engineer should know

Latency means "how long until I get an answer" — the delay before a single operation completes. Below are the canonical numbers (from the Jeff Dean / Peter Norvig lineage). *Do not memorize the exact figures.* Memorize the **orders of magnitude** — the powers of ten and the ratios. (One nanosecond, ns, is one billionth of a second; a millisecond, ms, is one thousandth.)

Operation	Approx. time	Relative to L1
L1 cache read (tiny on-chip memory)	0.5 ns	1×
Branch mispredict	5 ns	~10×
L2 cache read	7 ns	~14×
Mutex lock/unlock	25 ns	~50×
Main memory (RAM) read	100 ns	~200×
Send 1 KB over 1 Gbps network	~10 µs	~20,000×
Random 4 KB read, SSD (NVMe today ~10–70 µs)	~150 µs*	~300,000×
Round trip, same datacenter / cross-AZ	~0.5 ms	~1,000,000×
HDD (spinning disk) seek	~10 ms	~20,000,000×
Round trip, Virginia ↔ Ireland (AWS)	~68 ms	~130,000,000×
Round trip, California ↔ Netherlands	~150 ms	~300,000,000×

**The 150 µs is the 2012 figure for SATA SSD; modern NVMe SSDs do random 4 KB reads in roughly 10–70 µs. The shape of the table is unchanged.*

The shape that matters: roughly **every step down the storage hierarchy is about 10–100× slower than the one above it.** RAM is ~100× slower than L1 cache. SSD is ~1000× slower than RAM. A spinning disk seek and an intercontinental round trip are *millions* of times slower than L1.

Analogy: Multiply every time by a billion so one CPU cycle feels like one second. Then: L1 cache = 0.5 seconds. RAM = under 2 minutes. An SSD random read = ~1.7 days. A same-datacenter round trip = ~6 days. A disk seek = ~4 months. A California-to-Netherlands round trip = ~4.8 *years*. Suddenly "never call the network inside a loop" feels obvious — you'd never run an errand that takes years, a hundred times in a row.

Analogy: The storage hierarchy as a library. A fact in your head = registers/L1 (instant). A book open on your desk = RAM (a quick glance). A book on a shelf across the room = SSD (get up and walk). A book in another building = a spinning disk (drive across town). A book mailed from another country = a cross-region network call. Every step is dramatically slower than the last.

Bandwidth is not latency (the two-budget rule)

This trips up nearly everyone. **Bandwidth** is *how much* data you can move per second (megabytes per second). **Latency** is *how long* one round trip takes. They are *two separate budgets.*

You can *buy* bandwidth — add more or fatter pipes. You **cannot** buy latency below the speed of light. Light in fiber-optic cable travels at about two-thirds the speed of light, roughly 4.9 µs per kilometer one-way. So a 5,500 km link (Virginia to Ireland) can never beat about 27 ms one-way, ~54 ms for the round trip — and the measured AWS round trip is ~68 ms, i.e. physics plus a little routing overhead. No CDN, no upgrade, no money removes that 54 ms.

Analogy: A cargo ship versus a sports car. The ship has huge bandwidth (carries enormous cargo) but terrible latency (slow to arrive). The car carries little but arrives fast. Mailing a truck full of SSDs across the country can beat the internet on *bandwidth* — but it's hopeless on *latency*. Pick the right tool for the budget that's actually tight.

Best practice: To cut *latency*, cut *distance* — use edge servers, CDNs, or regional copies of your data closer to users. Buying a faster pipe only helps *bandwidth*. And to avoid latency pain, reduce the *number* of round trips: batch and pipeline requests instead of making fifty chatty back-and-forth calls.

The five resources, and the one skill that matters most

Every system runs against a fixed budget of physical resources. Engineering is deciding which to spend and which to conserve.

- **CPU** — compute cycles. Tight when you are "CPU-bound" (hashing, compression, parsing, ML inference). Add cores — but then you need concurrency (pillar 4) to use them.
- **Memory (RAM)** — fast, but small, expensive, and *volatile* (it's wiped when power is lost). Tight when your "working set" (the data you're actively using) doesn't fit. The penalty for overflowing is spilling to disk — a 1000×+ cliff.
- **Disk / storage** — durable, large, cheap, slow. Tight on capacity and especially on **IOPS** (random operations per second). *Sequential* access (reading data laid out in order) is far cheaper than *random* access (jumping around). This is the root reason databases love append-only logs and big ordered scans.
- **Network** — two budgets: bandwidth (scalable) and latency (physics-bound).
- **Time** — the meta-constraint, the one users actually feel. A latency budget like "respond within 200 ms" forces every other trade-off.

The core craft is this: **find the bottleneck resource first.** Optimizing anything else buys nothing.

Example: An API endpoint loops over 100 items and makes one database call per item. Each call is a ~50 ms network round trip: 100 × 50 ms = **5 seconds**, almost all of it spent waiting on the network. Doubling the CPU speed changes *nothing* — the CPU is idle, waiting. The fix is to replace 100 calls with *one batched query*. This is the famous "N+1 query" bug, and it's the single most common performance problem in real systems.

Common mistake: Optimizing a resource that isn't the bottleneck. A request stuck waiting on a 50 ms database call is not made faster by a faster CPU. **Measure first, then optimize the constraint you actually have.**

How the rest of this guide is built

This section was the map; the four pillars are the territory. We move in this order: **physics** (the latency numbers and the layer stack you just met) → **constraints** (the five resources) → **storage** (databases) → **coordination on one machine** (concurrency) → **coordination across machines** (distributed systems) → **the wire that connects them** (networking). Every later chapter is one variation of the same question: *how do we get good behavior out of a resource that is scarce, slow, or unreliable?* The latency table is the cheat sheet you'll mentally consult in every one of them.

Key takeaways:

- Fundamentals are durable because they're bounded by **physics and information theory**, not by changing frameworks — the latency table from 2012 is still right in 2026.
- Everything reduces to **four pillars** — databases, distributed systems, networking, concurrency — and a distributed database is all four at once.
- Picture the machine as **layers** (hardware → OS → process → network → data); reaching down or across costs time, and most slow code reaches across an expensive boundary inside a loop.
- Memorize **orders of magnitude**, not exact numbers: RAM ~100× L1, SSD ~1000× RAM, the network ~millions× L1.
- **Bandwidth and latency are separate budgets** — you can buy bandwidth, but latency below the speed of light is impossible; cut distance, not pipe size.
- The master skill is **finding the bottleneck resource** (CPU, memory, disk, network, time) before optimizing anything — and using these fundamentals to supervise, not trust, AI-generated systems.

How a Computer Runs Your Program: CPU, Memory, Processes & Threads

You write code in a language like Python, Java, or JavaScript. But a computer's brain — the **CPU** (Central Processing Unit, the chip that does all the calculating) — does not understand any of those languages. It understands only **machine code**: long strings of binary numbers (1s and 0s) that are instructions for one specific kind of chip. This section follows your program all the way down: how your text becomes machine code, how the CPU runs it, where the data lives, and how the operating system juggles many programs at once. Understanding this layer is what separates a developer who guesses at performance from one who knows.

2.1 From Source Code to Something the CPU Can Run

Your **source code** (the text you type) must be translated into machine code. There are three main strategies, and real languages mix them.

1. Compilation (AOT — ahead-of-time). A **compiler** (a program that translates code) turns your *whole* program into machine code *before* it runs. Languages like C, C++, Rust, and Go work this way. The result is a standalone **executable** file that is fast and needs no translator at run time — but it is tied to one type of chip and operating system. Many errors are caught during compiling, before the program ever runs.

2. Interpretation. An **interpreter** reads your program and runs it one statement at a time, translating as it goes. This is flexible and portable (the same code runs anywhere the interpreter exists), but slower, because the code is re-analyzed every time it runs.

3. Bytecode + a virtual machine. Languages like Java and Python first compile to **bytecode** — a compact, portable intermediate code that is *not* machine code for any real chip. Bytecode runs on a **virtual machine (VM)**, which is a piece of software that pretends to be a CPU. Java's `javac` tool produces `.class` bytecode that the **JVM** (Java Virtual Machine) executes. Python produces `.pyc` bytecode that **CPython** runs. This gives "write once, run anywhere."

4. JIT (just-in-time compilation). Modern VMs are clever. They start by interpreting bytecode, watch which functions run over and over (these are called "hot" paths), and then compile *just those* into native machine code *while the program is still running*. The JVM's HotSpot engine and every modern JavaScript engine (like Google's **V8**) do this. Because a JIT can see real data as the program runs, it can sometimes produce code faster than a plain ahead-of-time compiler.

```
C:       hello.c  --compiler+linker-->  native executable  --> CPU
Java:    Hello.java --javac--> .class bytecode --> JVM
                          (interprets, then JIT-compiles hot methods)
Python:  hello.py  --> .pyc bytecode  --> CPython interprets
```

Key takeaway: Everything ends up as machine code the CPU executes. The only question is *when* the translation happens — before running (AOT), during running (JIT), or piece-by-piece every time (interpretation).

Common mistake: Believing "compiled = fast, interpreted = slow" as an absolute rule. Modern JIT engines (V8, JVM HotSpot) compile hot code to native machine code at run time and can rival or beat naive compiled code. Python is slow for separate reasons (its dynamic typing and a lock called the GIL), not simply because "it's interpreted."

2.2 The CPU and the Fetch–Decode–Execute Cycle

At its heart, a CPU does one thing in a loop, forever:

1. **Fetch** — read the next instruction from memory. The CPU keeps the address of that instruction in a special slot called the **program counter (PC)**, also called the instruction pointer.
2. **Decode** — work out what the instruction means and what data it needs.
3. **Execute** — the **ALU** (Arithmetic Logic Unit, the part that does math and logic) performs the operation; the result goes into a register or memory.
4. **Advance the PC** to the next instruction (or jump elsewhere), then repeat.

This loop is driven by the **clock**, an electronic pulse that ticks billions of times per second. A 3 GHz CPU ticks 3 billion times per second, so one tick (cycle) takes about 0.3 nanoseconds. (A **nanosecond** is one-billionth of a second.)

Real CPUs are far cleverer than "one instruction at a time." **Pipelining** overlaps the fetch/decode/execute stages of several instructions, like an assembly line. **Superscalar** chips run several instructions in the same cycle. **Out-of-order** and **speculative** execution, helped by **branch prediction** (guessing which way an `if` will go), keep the assembly line full. But fetch-decode-execute is the correct mental model to start from.

Registers — the fastest storage that exists

Registers are a tiny set of storage slots *inside* the CPU itself. They are the fastest memory there is, read in well under one clock cycle. An x86-64 chip has about **16 general-purpose 64-bit registers** (with names like RAX and RBX), plus the program counter, a stack pointer, and flag registers. The crucial fact: **all computation happens in registers.** To add two numbers, the CPU loads them from memory into registers, adds, and stores the result back. The CPU spends much of its life shuttling data between slow memory and these few fast registers.

2.3 The Memory Hierarchy — Why Locality Decides Speed

There is one unbreakable trade-off in hardware: **fast memory is small and expensive; large memory is slow and cheap.** So computers stack layers, each bigger and slower than the one above it.

```
       /\          Registers   <1 ns     ~hundreds of bytes
      /  \         L1 cache     ~1 ns     ~32-64 KB per core
     / fast\       L2 cache     ~3-5 ns   256 KB-2 MB per core
    /  small\      L3 cache     ~10-20 ns tens of MB (shared)
   /----------\    Main memory  ~100 ns   gigabytes (RAM)
  /  slow big  \   SSD          ~150 us   hundreds of GB-TB
 /--------------\  Hard disk    ~10 ms    terabytes
```

The exact numbers vary by machine; **the ratios are what matter**. RAM is roughly **100–200× slower than L1 cache**. An SSD is about **1,000× slower than RAM**. A spinning hard disk seek is about **100,000× slower than RAM**.

Analogy: Scale every time up by one billion so it lands in human terms. One CPU cycle becomes 1 second. Reaching L1 cache: a few seconds. Reaching RAM: about 6 minutes. Reading from an SSD: about 2 days. A hard-disk seek: about a year. Suddenly "just read it from disk" sounds as crazy as it really is to the CPU.

Caches (L1, L2, L3) are small, very fast memory (a type called SRAM) that hold copies of data the CPU recently used or will likely use. When the CPU needs data it checks L1, then L2, then L3, then RAM. Finding it early is a **cache hit**; not finding it is a **cache miss**, which stalls the CPU while it waits for a slower layer. L1 and L2 are usually *per core* (each core has its own); L3 is usually *shared* across all cores. Caches move data in fixed-size chunks called **cache lines** — typically **64 bytes** — so touching one byte pulls in its 64-byte neighborhood.

Caches work because of **locality**, the single most important performance idea in this section:

- **Temporal locality** — data you used recently, you will probably use again soon. (So keep it cached.)
- **Spatial locality** — data *near* what you just used, you will probably use soon. (So pull in whole cache lines.)

Example: Summing every number in a big 2D grid. If you walk it *row by row*, you visit memory in order — each 64-byte cache line is fully used before moving on. Fast. If you walk it *column by column*, you jump far between each access, wasting most of every cache line you load. Same `O(n×n)` amount of work, but the column version can be 5–10× slower purely from poor spatial locality.

Common mistake: Treating RAM as "fast" and all memory access as equal cost. In textbook algorithm analysis memory is uniform-cost, but on real hardware a cache miss to RAM can dominate the runtime. Two programs with identical Big-O can differ 10× from locality alone. Cache-friendly data layout (arrays beat pointer-chasing linked lists) is a real, measurable win.

2.4 Virtual Memory and Paging

Each running program is given the illusion that it owns one huge, private, continuous block of memory — its **virtual address space**. In reality, physical RAM is shared among many programs and scattered into fragments. The operating system, together with a hardware unit on the CPU called the **MMU** (Memory Management Unit), translates the **virtual address** a program sees into the real **physical address** in RAM.

- Memory is divided into fixed-size **pages** on the virtual side and matching **frames** on the physical side — almost always **4 KB** each (with optional larger "huge pages" of 2 MB or 1 GB).
- Each process has a **page table** that maps each virtual page to a physical frame.
- Walking the page table on every access would be slow, so the MMU keeps a **TLB** (Translation Lookaside Buffer) — a small hardware cache of recent translations. A **TLB hit** is instant; a **TLB miss** forces a slower walk of the page table.
- If a page isn't in RAM at all, the MMU raises a **page fault**; the OS loads the page (maybe from disk **swap** space) and retries. When this happens constantly it's called **thrashing**, and performance collapses.

Example (address translation): Take a 32-bit virtual address with 4 KB pages. Since 4 KB = 2^{12} bytes, the bottom **12 bits** are the *offset* inside the page, and the top **20 bits** are the *page number*. The MMU looks up the 20-bit page number in the TLB/page table to find a physical frame, then glues the unchanged 12-bit offset onto it. The offset never changes — only the page-number part gets remapped.

Virtual memory gives three big wins: **isolation** (one process literally cannot read or corrupt another's memory — a security and stability wall), the **illusion of more memory** than the physical RAM, and a **clean, uniform address space** so every program can be written as if it owns the machine.

Common mistake: Confusing a virtual address with a physical one. The pointer value your program prints is virtual. The same virtual address in two different processes points to *different* physical RAM. Never assume two processes share memory just because their addresses look the same.

2.5 Processes vs Threads

A **process** is a running program with its own isolated virtual address space. Two processes cannot see each other's memory by default; to communicate they must use explicit channels (pipes, network sockets, or shared-memory segments) — together called **IPC** (Inter-Process Communication). Each process has its own code, heap, global variables, open files, and at least one thread.

A **thread** is a single line of execution *inside* a process. All threads in one process **share the same memory** — the same heap, globals, and code — but each thread has its **own stack, own registers, and own program counter**. That shared memory makes threads cheap to create and fast to communicate. It is also exactly what makes concurrency bugs possible.

```
PROCESS  (one private address space)
+-----------------------------------------+
|  Code  |  Globals  |  Heap   (SHARED)     |
|-----------------------------------------|
|  Thread 1:  own stack + own registers    |
|  Thread 2:  own stack + own registers    |
+-----------------------------------------+
```

Analogy: A process is a private kitchen with its own pantry no other kitchen can touch. The threads are several cooks sharing that one kitchen and pantry (shared memory), but each cook has their own cutting board (own stack). The head chef rotating cooks on and off the single stove is the scheduler doing context switches.

Common mistake: Assuming threads in one process have separate memory. They share the heap, globals, and code — that's the whole point and the whole danger. Two threads writing the same variable without coordination causes bugs. Best practice: protect shared changing data with locks or atomic operations, or avoid sharing it at all.

2.6 Stack vs Heap

Inside a process's address space, two regions grow toward each other.

```
 high addresses
 +------------------+
 |   Stack          |  grows DOWN  v   (function call frames)
 |        |         |
 |        v         |
 |   (free gap)     |
 |        ^         |
 |        |         |
 |   Heap           |  grows UP    ^   (dynamic objects)
 +------------------+
 |   Globals        |
 |   Code (text)    |
 +------------------+
 low addresses
```

Aspect	Stack	Heap
Stores	Function call frames: parameters, local variables, return address	Dynamically sized, longer-lived data (objects, big buffers)
Speed	Very fast — just move the stack pointer	Slower — an allocator must find free space
Freed by	Automatically, when the function returns	Manually (`free`) or by a garbage collector
Size	Small & fixed (often ~1–8 MB)	Large, flexible
Typical failure	Stack overflow (e.g. infinite recursion)	Memory leaks, use-after-free, fragmentation

Each function call **pushes** a **stack frame**; returning **pops** it. A **garbage collector (GC)** is a background helper (in Java, Python, JavaScript) that automatically frees heap memory no longer used.

Common mistake: Putting huge or unknown-size data on the stack causes a **stack overflow** (the small stack runs out of room). And assuming heap allocation is free — it isn't; it's slower and can fragment. Prefer the stack for small, short-lived, fixed-size data.

2.7 The Scheduler, Context Switches, and User vs Kernel Mode

One CPU core runs exactly one thread at any instant. The **scheduler** (part of the OS) rapidly rotates many threads onto the available cores, giving each a short **time slice** before switching to the next. Because it switches thousands of times a second, everything *appears* to run at once. This is **preemptive multitasking** — the OS can pause ("preempt") a thread without asking it.

Swapping one thread for another is a **context switch**: the CPU saves the current thread's registers and program counter, then loads another's. A switch *between threads of the same process* is cheap, because the memory map (and the TLB) stays the same. A switch *between different processes* is expensive: the address space changes, so the **TLB must be flushed** (its cached translations are now wrong) and the caches go cold. Context switches are pure overhead — useful work stops while they happen.

User mode vs kernel mode are hardware-enforced privilege levels. Your code runs in **user mode**, with no direct access to hardware or other processes' memory. The OS **kernel** (its trusted core) runs in **kernel mode** with full access. The only controlled doorway between them is a **system call** — a request like "open this file" or "send these bytes on this socket." A system call traps into the kernel, switches to kernel mode, does the privileged work, and returns. System calls are relatively expensive, which is why fast code *minimizes* them (by batching and buffering I/O).

Best practice: Treat context switches and system calls as costing real time. Don't make thousands of tiny syscalls when one batched call works. Right-size thread pools to the workload — more threads is not automatically faster.

2.8 Why This Is the Foundation for Concurrency

Two facts from this section combine into the central challenge of all concurrent programming. First, the scheduler interleaves threads **unpredictably** — you cannot know exactly when one thread pauses and another resumes. Second, threads in a process **share memory**. When two threads read-modify-write the same shared data without coordination, the unlucky interleaving produces a wrong result. This is a **race condition**, and it's why locks and atomic operations exist.

Common mistake: Confusing concurrency with parallelism, and assuming "more threads = more speed." *Concurrency* is interleaving many tasks (even on one core via the scheduler). *Parallelism* is running them truly at the same time on multiple cores. Past the core count — or with heavy lock contention or constant context switching — adding threads makes things *slower*.

Common mistake (false sharing): Two threads that update *different* variables which happen to sit on the same 64-byte cache line will fight over that line, silently destroying multi-thread performance even though they never logically share data. Awareness of cache lines prevents this.

Key takeaways:

- All code becomes machine code; the difference between compiled, interpreted, bytecode+VM, and JIT is only *when* that translation happens — and modern JITs can be very fast.
- The CPU runs a fetch–decode–execute loop on data held in a few ultra-fast registers; real chips pipeline and run instructions out of order to stay busy.
- Memory is a hierarchy where ratios rule: RAM is ~100× slower than L1 cache, disk ~100,000× slower than RAM. Locality-friendly code beats locality-hostile code even at the same Big-O.
- Virtual memory gives every process an isolated, clean address space; the MMU + TLB translate virtual pages to physical frames, with page faults and swap as the slow fallback.
- A process is isolated memory; threads share that memory but keep their own stack and registers — fast to communicate, but the source of race conditions.
- The scheduler time-slices threads; context switches and system calls cost real time (process switches flush the TLB), so minimize them and match thread count to cores.

Concurrency & Parallelism: Doing Many Things at Once

Modern programs rarely do just one thing at a time. A web server answers thousands of users at once. Your phone plays music while downloading a file while checking for messages. This section explains how computers juggle many tasks, the bugs that show up when they do, and the tools that keep things correct. We start from the very basics and build up.

1. The Big Distinction: Concurrency vs. Parallelism

People use these two words as if they mean the same thing. They do not. Getting this clear early will make everything else easier.

Concurrency is about *structure*: organizing a program so that many tasks can *make progress over overlapping time periods*. "Make progress over overlapping time" means each task moves forward a little, then the next one does, then back to the first. They are all "in flight" at the same time, but at any single instant only one might actually be running. A computer with a single CPU core (a "core" is one processing unit that runs instructions) does this by **time-slicing** — switching between tasks so fast it looks simultaneous.

Parallelism is about *execution*: literally running multiple tasks *at the exact same instant*. This requires multiple cores. Two cores can each run a task in the same nanosecond.

> **Analogy:** Imagine a coffee shop. **One barista** handling two orders — start one espresso, while it brews go steam milk for the other, then back — is *concurrent but not parallel*. Both orders progress, but only one pair of hands is moving at any instant. Add a **second barista** and now both orders are worked on at the same moment — that is *parallel*. One core = one barista (concurrency by switching). Two cores = two baristas (real parallelism).

Rob Pike, the designer of the Go programming language, put it memorably: *"Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once."* Concurrency is how you **structure** a program; parallelism is a **property of the hardware** running it.

> **Key takeaway:** Concurrency *enables* parallelism but does not *require* it. You can be concurrent on one core (time-slicing) and parallel without elaborate concurrency structure (splitting a giant sum across cores). They are related, not identical.

2. Why Bother? It Depends on What's Slowing You Down

The right concurrency tool depends entirely on what your program spends its time doing. There are two flavors of work:

- **I/O-bound work:** "I/O" means input/output — reading a disk, calling a network service, querying a database. The bottleneck is *waiting*. While the program waits for a database to reply (say 100 milliseconds), the CPU sits idle doing nothing. Concurrency wins big here: during the wait, let *another* task use the CPU. This works even on a single core.
- **CPU-bound work:** The bottleneck is *computation* — hashing passwords, resizing images, crunching numbers. There is no waiting to hide; the CPU is busy the whole time. Concurrency alone gives you nothing here. To go faster you need **true parallelism**: more cores doing real work simultaneously.

Aspect	I/O-bound	CPU-bound
Bottleneck	Waiting (disk, network, DB)	Calculation
CPU busy?	Mostly idle, waiting	Fully busy
Best tool	Async / event-loop or threads	Multiple processes / parallel threads
Helped by more cores?	Not much — hiding waits is enough	Yes — that's the whole point

Common mistake: Assuming "concurrent code automatically uses all my cores." It does not. Concurrency is a structure; whether it runs in parallel depends on the runtime and hardware. Likewise, do not assume a single core means "no concurrency" — time-slicing gives concurrency on one core.

3. Three Ways to Do Many Things at Once

There are three common models. Define each carefully.

- **Processes:** A process is an independent running program with its *own private memory*. Two processes cannot accidentally corrupt each other's data because they are isolated. If one crashes, the others survive. The downside: they are *heavyweight* (slow to create) and talking between them needs **IPC** (inter-process communication — pipes, sockets, shared-memory segments), which is more work.
- **Threads:** A thread is an execution stream *inside* a process. Multiple threads in one process *share the same memory*. They are lightweight and fast to create, and they communicate just by reading/writing shared variables. But that shared memory is exactly what causes the bugs we'll see next. The operating system's **scheduler** (the part that decides which thread runs) can **preempt** — interrupt — a thread at almost *any* instruction.
- **Async / event-loop:** A *single* thread runs an **event loop** — a loop that picks up ready tasks and runs them a piece at a time. Tasks *cooperatively* give up control at marked pause points (often the keyword `await`). Because a task is never interrupted mid-step, there are far fewer races. But one greedy task that never yields freezes everything.

How Node.js does it (a concrete real example)

Node.js (a JavaScript runtime) runs your JavaScript on a *single* thread with an event loop. Underneath sits a C library called **libuv**. For network I/O, libuv asks the operating system's native async facilities (`epoll` on Linux, `kqueue` on macOS, `IOCP` on Windows) to notify it when data is ready — no extra threads needed. But some operations can't be done async by the OS (file system access, `crypto` hashing, DNS lookups). For those, libuv uses a small **thread pool of 4 threads by default** (you can change it with the `UV_THREADPOOL_SIZE` setting). When a worker finishes, its callback is queued back onto the event loop. So "single-threaded Node" quietly has worker threads for the blocking jobs.

4. The Core Danger: Race Conditions

A **race condition** is when the correctness of your program depends on the *timing* of how threads interleave — and some interleavings produce wrong answers. The name comes from threads "racing" each other to touch shared data.

The classic example everyone must understand is the **lost update**. Consider `counter++` (add one to a shared counter). It *looks* like one step, but the CPU actually does three: **READ** the value from memory, **MODIFY** it (add 1) in a register, and **WRITE** it back. With two threads and `counter = 0`:

```
Thread A          Thread B          counter in memory
READ  (sees 0)                      0
                  READ  (sees 0)    0
ADD -> 1                            0
                  ADD -> 1          0
WRITE 1                             1
                  WRITE 1           1   <-- should be 2!
```

Thread B read the old value (0) *before* A wrote its result. One increment vanished. Run two threads each incrementing a shared counter 1,000,000 times and you will get a final number *less than* 2,000,000 — and a different number each run.

- The **critical section** is the code that touches shared data and must not be interleaved (here, the read-modify-write).
- **Mutual exclusion** is the guarantee that *only one thread at a time* may be inside the critical section.

> **Common mistake:** Assuming `counter++`, `i++`, `list.append`, or a dictionary update is "atomic" (indivisible). It usually is not. Before teaching anyone the fix, reproduce the lost update — seeing <2,000,000 makes it real.

5. The Toolbox: Synchronization Primitives

These are the low-level tools that keep shared state safe.

- **Lock / Mutex** ("mutual exclusion"): a thread calls `acquire()` before the critical section and `release()` after. Only the holder proceeds; everyone else waits. Wrap `counter++` in a lock and the result is always 2,000,000.

- **Semaphore:** a counter that permits up to *N* threads at once. A mutex is just a semaphore with N=1. Use it to cap concurrency, e.g. "at most 10 simultaneous database connections."
- **Atomic operations:** hardware-guaranteed indivisible operations, such as *compare-and-swap (CAS)* or atomic fetch-add. An atomic increment can't be split apart, so it fixes the counter *without* a lock — faster for simple cases.
- **Memory barriers / visibility:** modern CPUs cache values in per-core caches and reorder instructions for speed. So even with no torn write, one thread's update may not be *visible* to another. A **memory barrier** (or the guarantees built into a language's memory model) forces the update to be published across cores. This is why Java has `volatile`, C++ has `std::atomic`, and Go ships a race detector. Locks and atomics also act as barriers.

Common mistake: Assuming a write in one thread is instantly seen by another without a lock, atomic, `volatile`, or barrier. Visibility is not free.

6. The Three Classic Failures

Deadlock is when two or more threads each hold a resource the other needs, and nobody lets go — frozen forever. It requires all **four Coffman conditions** at once:

1. **Mutual Exclusion** — a resource can't be shared.
2. **Hold and Wait** — a thread holds one resource while waiting for another.
3. **No Preemption** — resources are only given up voluntarily.
4. **Circular Wait** — a cycle of waiting, P1 waits on P2 waits on … waits on P1.

Break *any one* and deadlock becomes impossible.

```
Thread A holds Lock1, wants Lock2
Thread B holds Lock2, wants Lock1

     A --waits for--> Lock2 --held by--> B
     ^                                   |
     |                                   v
   Lock1 <--held by-- A   B --waits for--> Lock1
        (a circle = circular wait = stuck)

FIX: everyone always grabs Lock1 BEFORE Lock2.
     No circle can form. Deadlock gone.
```

Best practice: The most practical deadlock fix is **lock ordering** — define one global order for acquiring locks and follow it everywhere. This breaks the circular-wait condition.

Livelock: threads are *not* blocked — they are busy running and changing state — but make *no real progress*. Often caused by overly polite retry/back-off logic.

Analogy: Two people meet in a narrow hallway. Both step left to let the other pass. Both step right. Both step left again. They keep moving, never colliding, and never get through. That is livelock — motion without progress.

Starvation: a thread is perpetually denied a resource because others keep cutting ahead (e.g., a low-priority thread that never gets scheduled, or a writer blocked forever by a stream of readers). The fix is *fairness policies* — fair locks that take turns.

7. Amdahl's Law: The Hard Ceiling on Speedup

Adding more cores does not give unlimited speedup. **Amdahl's Law** tells you the maximum. Let **p** be the fraction of your program that can run in parallel, and **s** the number of processors:

```
Speedup = 1 / ( (1 − p) + p/s )
```

As *s* grows toward infinity, `p/s` shrinks to zero, so speedup approaches `1 / (1 − p)`. The *serial* (non-parallel) part sets a hard cap.

Example: Suppose 90% of your work can be parallelized (p = 0.9). The maximum speedup, even with *infinite* cores, is 1 / (1 − 0.9) = **10×**. Never more. If you push to 95% parallel, the cap becomes 1 / 0.05 = **20×**. That last 5–10% of serial code is what limits you — not the number of cores.

The optimistic counterpoint is **Gustafson's Law**: in the real world, when you have more cores you usually tackle a *bigger* problem, and the parallel fraction grows with problem size. So practical scaling can beat Amdahl's fixed-size pessimism.

Common mistake: Throwing cores at a problem without profiling. Once the serial fraction dominates, extra cores buy almost nothing. Shave the sequential code first.

8. Higher-Level Models (Where Modern Code Lives)

Hand-written locks are error-prone, so most modern systems prefer safer abstractions:

- **Message passing / Actors:** no shared memory at all. Each "actor" has private state and a mailbox, and they only communicate by sending immutable messages. An actor handles one message at a time, so its state needs no locks. (Erlang, Elixir, Akka.) This eliminates whole classes of races.
- **CSP / Channels (Go):** "Communicating Sequential Processes." Lightweight **goroutines** (tiny concurrent functions) pass values through **channels** instead of touching shared memory. Go's motto: *"Don't communicate by sharing memory; share memory by communicating."* A channel both moves data and synchronizes — a send waits until a receiver is ready.
- **async / await event loops** (Node, Python `asyncio`): cooperative single-threaded concurrency, excellent for I/O-bound work. `await` yields control during a wait so other tasks run.

- **Thread pools:** reuse a fixed set of worker threads instead of spawning a new one per task. Tasks go on a queue; idle workers pick them up. Creating threads is expensive and unbounded threads exhaust memory.
- **Producer-consumer + backpressure:** the backbone of server work-dispatch.

```
[Producers] ---> [ bounded thread-safe queue ] ---> [Consumers]

queue FULL  => producers block  (this is "backpressure")
queue EMPTY => consumers block  (wait for work)
```

Backpressure means the queue's fixed size automatically slows fast producers when consumers can't keep up — a built-in safety valve.

Common mistake: Running a CPU-heavy or blocking call inside an async event loop without yielding. It freezes the entire loop and starves every other task. Keep the loop's tasks short and I/O-focused; offload heavy compute to a worker.

9. Thread-Safety & Immutability — the Cheapest Win

Code is **thread-safe** if it behaves correctly under concurrent access without extra synchronization. The cheapest way to get there is **immutability**: if data never changes after it is created, there is nothing to race on — no locks needed, ever. This is why functional and message-passing styles scale so well.

Best practice: Prefer immutable data and copies over shared mutable state. The race you don't create is the bug you never debug.

10. Python's GIL (a Real-World Wrinkle)

CPython (the standard Python) has historically had a **Global Interpreter Lock (GIL)** — a single lock that lets only one thread run Python bytecode at a time. So Python *threads* do not give CPU-bound parallelism; for that you use the `multiprocessing` module (separate processes, separate memory, no shared GIL). But threads *do* help I/O-bound work, because the GIL is released while a thread waits on I/O.

Current status (2025/2026): PEP 703 (the "free-threaded," no-GIL proposal) was accepted in 2023. Python 3.13 (October 2024) shipped an *experimental* free-threaded build. Python 3.14 (2025) moved free-threading to officially *supported* status (via PEP 779). It is still opt-in, not the default, carries a roughly 5–10% single-thread overhead, and default-on remains years away.

Common mistake: Using Python threads for CPU-bound work and expecting a speedup. The GIL serializes bytecode. Use `multiprocessing` or the free-threaded build instead.

11. Tying It Together: A Web Server With 10,000 Requests

A server receiving 10,000 simultaneous requests, each waiting about 100ms on a database, is **I/O-bound** (mostly waiting). Three approaches:

- **Thread-per-request:** spawn 10,000 threads. It works, but threads are heavyweight — that's a lot of memory and constant context-switching (the cost of the OS swapping threads on the CPU).
- **Thread pool:** a bounded set of workers pulls requests off a queue (producer-consumer). Stable memory, controlled concurrency.
- **Async event loop:** one or a few threads juggle all 10,000 using non-blocking I/O. This scales far better when the work is mostly waiting.

This is the famous **C10k problem** (handling 10,000 concurrent connections) and why async-first servers — nginx, Node.js, Go — dominate I/O-heavy workloads.

Key takeaways:

- Concurrency is structure (dealing with many tasks); parallelism is execution (running them at once). Concurrency enables but doesn't require parallelism.
- Match the tool to the bottleneck: async/event-loop or threads for I/O-bound (hide waiting), multiple processes/parallel threads for CPU-bound (need more cores).
- Shared mutable memory causes race conditions; `counter++` is a read-modify-write and can lose updates. Protect critical sections with locks, atomics, or higher-level models — and immutability avoids the problem entirely.
- Deadlock needs all four Coffman conditions; break one (lock ordering breaks circular wait). Watch also for livelock (busy, no progress) and starvation (perpetually denied).
- Amdahl's Law caps speedup at $1/(1-p)$: a 90%-parallel program tops out at 10× no matter how many cores. Profile and shrink the serial part first.
- Prefer channels/actors/queues over hand-rolled locks; bound your thread pools; enable race detectors in CI; remember Python's GIL serializes CPU-bound threads (free-threading is supported but not default as of 3.14).

Databases I — Relational Databases, SQL & ACID

Almost every serious application needs to **store data** — orders, users, payments, messages — and get it back later, correctly, even when many people use the app at the same time. The tool that does this job is a **database**. In this section we build up from "what is a database?" to the deep guarantees (called **ACID**) that let a bank move your money without ever losing a cent. Take it slowly; each idea builds on the last.

1. What is a database, and why not just use files?

A **database** is a structured system for storing, searching, and safely changing data — especially when many users touch it at once. A **DBMS** (Database Management System) is the software that runs the database. Popular ones are **PostgreSQL**, **MySQL** (and its cousin MariaDB), **SQLite**, SQL Server, and Oracle.

You could, in theory, store your orders in plain *files* — a CSV (comma-separated values) file, or a folder of JSON files. People try this and it falls apart quickly. Here is why a real database wins on five fronts:

- **Concurrency control** — "concurrency" means many things happening at the same time. If two parts of your program write to the same file at once, they can overwrite each other and corrupt the file. A database safely interleaves thousands of simultaneous writers.
- **Querying** — a "query" is a question you ask of the data. With files, "all orders over $100 placed last week by customers in Texas" means writing a loop by hand. A database answers it in one line of SQL.
- **Integrity** — "integrity" means the data stays valid and consistent. A database can refuse to save an order for a customer that doesn't exist, or two users with the same ID. Files happily save garbage.
- **Crash safety** — if the power dies halfway through writing a file, you get a half-written, broken file. A database guarantees a write either fully happened or didn't happen at all.
- **Indexes** — an "index" is a lookup structure (like a book's index) that finds one row fast. Without it, finding one row among 10 million means reading all 10 million. We cover indexes deeply in a later section.

> **Analogy:** A flat file is a single shared notebook on a desk — fine for one person, chaos when ten people grab it at once and scribble. A DBMS is a well-run library: a librarian (the DBMS) checks books in and out, keeps a catalog so you find any book instantly, and never lets two people write on the same page at the same time.

2. The relational model: tables, rows, columns, keys

A **relational database** organizes data into **tables**. Think of a table as a spreadsheet:

- A **column** (also called a field) is a vertical category, like `email` or `price`. Each column has a fixed **data type** — the kind of value it holds (a number, text, a date).
- A **row** (also called a record) is one horizontal entry — one customer, one order.

Two special kinds of columns hold the model together:

- **Primary key (PK)** — a column (or group of columns) that *uniquely identifies* each row. It must be **unique** (no two rows share it) and **not null** ("null" means "no value" — a PK can never be empty). Most tables use a **surrogate key**: a meaningless-but-stable ID, like an auto-incrementing number (1, 2, 3…) or a UUID (a long random unique string).
- **Foreign key (FK)** — a column whose value must match a primary key in *another* table. This enforces **referential integrity**: the database won't let you save an order whose `customer_id` points at a customer that doesn't exist. You also control what happens when the referenced row is deleted — `ON DELETE RESTRICT` blocks the delete, `ON DELETE CASCADE` deletes the children too.

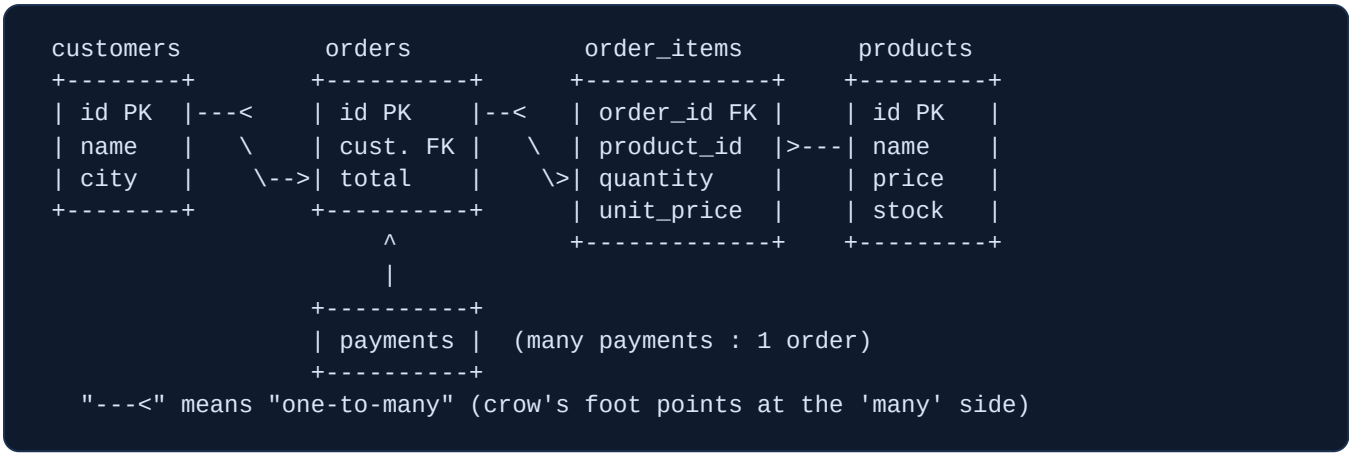
> **Common mistake:** Using a real-world, changeable value (like an email address) as the primary key. Emails change. When they do, every row pointing at that key breaks. Use a stable surrogate key instead and let email be a normal `UNIQUE` column.

Relationships: how tables connect

Cardinality means "how many on each side."

Type	Meaning	How to build it
1:1	One row maps to exactly one row elsewhere (a `user` and their `user_profile`).	A foreign key that is also marked `UNIQUE`.
1:many	The common case: one `customer` has many `orders`.	Put the FK on the *many* side: `orders.customer_id`.
many:many	An `order` has many `products`, and a `product` appears on many orders.	You need a **join table** (next paragraph).

A single foreign key cannot represent **many:many**. You need a third table — a **join table** (also called a junction or bridge table). For orders and products, that's `order_items(order_id, product_id, quantity, unit_price)`, holding a FK to each side. Crucially, the join table also stores facts *about the relationship itself* — here, the quantity and the price paid.



3. SQL basics: asking and changing

SQL = Structured Query Language, the language for talking to a relational database. The four core **DML** (Data Manipulation Language) statements:

- `SELECT name, total FROM orders WHERE total > 100 ORDER BY total LIMIT 10;` — **read** data.
- `INSERT INTO orders (customer_id, total) VALUES (5, 99.50);` — **create** a row.
- `UPDATE orders SET total = 120 WHERE id = 10;` — **change** rows.
- `DELETE FROM orders WHERE id = 10;` — **remove** rows.

> **Common mistake (career-ending edition):** Running `UPDATE` or `DELETE` *without a* `WHERE` *clause*. `DELETE FROM orders;` deletes **every order in the table**. `UPDATE accounts SET balance = 0;` zeroes out everyone. The `WHERE` says which rows; forget it and the change hits all of them. Always write the `WHERE` first.

JOINs: combining tables

A **JOIN** stitches rows from two tables together using a matching condition (the `ON` clause).

Join type	What it returns
INNER JOIN	Only rows that have a match on *both* sides.
LEFT JOIN	All rows from the left table, plus matches from the right (NULLs where there's no match).
RIGHT JOIN	Mirror image: all rows from the right table.
FULL OUTER JOIN	All rows from both sides, matched where possible.

Basic inner join — customers and their orders:

```
SELECT c.name, o.total FROM customers c JOIN orders o ON o.customer_id = c.id;
```

The classic **LEFT JOIN** use case — find customers who have *never* ordered. Keep all customers, attach orders where they exist, then keep only the rows where no order matched (the order columns came back NULL):

```
SELECT c.name FROM customers c LEFT JOIN orders o ON o.customer_id = c.id WHERE o.id IS NULL;
```

> **Common mistake:** Using `INNER JOIN` when you needed `LEFT JOIN`. An inner join *drops* unmatched rows — so customers with zero orders silently vanish from your report, and you never notice the gap.

Aggregation: collapsing many rows into a summary

Aggregate functions reduce many rows to one number: `COUNT` (how many), `SUM` (total), `AVG` (average), `MIN`, `MAX`. `GROUP BY` produces one output row per group. Revenue per customer, keeping only big spenders:

```
SELECT customer_id, SUM(total) AS revenue FROM orders GROUP BY customer_id HAVING SUM(total) > 1000;
```

Notice two filters that look similar but run at different times:

- `WHERE` filters **individual rows before grouping**. It cannot mention `SUM` or `COUNT` because the groups don't exist yet.
- `HAVING` filters **groups after grouping**. This is where you say "only groups whose total exceeds 1000."

> **Key takeaway:** `WHERE` runs first on raw rows; `HAVING` runs last on grouped results. To exclude refunded orders *and* keep only big spenders, you use both: `WHERE status <> 'refunded' ... GROUP BY ... HAVING SUM(total) > 1000`.

4. Schema and data types

The **schema** is the blueprint of your database — the tables, their columns, the types, and the rules — defined with `CREATE TABLE`. Picking the right **data type** matters for both correctness and storage:

- `INTEGER` / `BIGINT` — whole numbers.
- `NUMERIC(p,s)` / `DECIMAL(p,s)` — exact decimal numbers. **Use this for money.** (`p` = total digits, `s` = digits after the decimal point.)
- `VARCHAR(n)` / `TEXT` — text.
- `BOOLEAN` — true/false.
- `DATE`, `TIMESTAMP`, `TIMESTAMPTZ` — dates and times; prefer the timezone-aware `TIMESTAMPTZ`.
- `UUID` — a long random unique ID.

- `JSON` / `JSONB` (PostgreSQL) — for flexible, semi-structured data.

> **Common mistake — money in FLOAT:** A `FLOAT` / `DOUBLE` stores numbers in binary, and it *cannot* represent values like 0.10 exactly. In floating point, `0.1 + 0.2` comes out as `0.30000000000000004`. Over thousands of orders these tiny errors accumulate into real, visible discrepancies in totals and payouts. Store money in `NUMERIC(10,2)`, which holds 0.10 + 0.20 = 0.30 exactly.

Constraints are rules the database enforces so bad data *can never be saved*:

- `NOT NULL` — value is required.
- `UNIQUE` — no duplicates (e.g. one account per email).
- `CHECK (total >= 0)` — a custom condition that must hold (e.g. stock can't go negative).
- `DEFAULT` — a value used when none is given (e.g. `status DEFAULT 'pending'`).

> **Best practice:** Push business rules into constraints, not just app code. A `CHECK (stock >= 0)` guarantees you can never oversell even if a future bug forgets to check. The database is your last line of defense.

5. Normalization: removing harmful duplication

Normalization is the process of organizing columns so data isn't pointlessly duplicated. Duplication causes **update anomalies** — you change a customer's city in one row but forget the other ten, and now the data disagrees with itself. There are three main "normal forms," each building on the last. We'll evolve one messy table step by step.

Start with this badly designed table:
`orders(id, customer_id, customer_city, product_id, product_name, quantity)`

1NF — atomic values, no repeating groups

"Atomic" means each cell holds *one* value. No comma-lists like `"red,blue,green"` in a single column, and no repeating columns like `phone1, phone2, phone3`. If our order tried to cram several products into one row as a list, 1NF forces us to give each product its own row (or its own line-item record).

> **Common mistake:** Stuffing `"red,blue"` into one cell. Now you can't easily filter "orders containing blue," can't index it, and can't count colors. It quietly destroys your ability to query.

2NF — no partial dependency on part of a composite key

This only matters when the primary key is **composite** (made of more than one column). Every non-key column must depend on the *whole* key, not just part of it. In a line-items table keyed by `(order_id, product_id)`, the column `product_name` depends only on `product_id` — half the key. That's a **partial dependency**. Fix: move `product_name` out to a `products` table.

3NF — no transitive dependency

A **transitive dependency** is when a non-key column depends on *another non-key column* instead of the key. In `orders(id, customer_id, customer_city)`, `customer_city` really depends on `customer_id`, not on the order. Fix: move `customer_city` to the `customers` table, where it belongs.

Key takeaway — the mnemonic: Every non-key column must depend on *"the key, the whole key, and nothing but the key, so help me Codd."* "The key" = 1NF (there is a key), "the whole key" = 2NF (no partial dependency), "nothing but the key" = 3NF (no transitive dependency).

Denormalization: breaking the rules on purpose

Denormalization means deliberately adding redundancy back to make *reads faster* — for example storing a precomputed `order_total` instead of summing line items on every page load. The trade-off: you must keep the copies in sync on every write. Choose it deliberately, only where read speed truly matters.

Key takeaway — snapshotting is NOT a violation: When an order copies the product's name and price *at the moment of purchase*, that is correct, not a normalization bug. The snapshot is a genuinely different fact ("price paid on June 1") than the live catalog price ("price today"). If you didn't snapshot, editing a product's price tomorrow would silently rewrite every past invoice — a serious accounting error.

Common mistake: Over-normalizing a heavy reporting query into 12 joins (slow), *or* over-denormalizing and forgetting to keep the duplicate copies in sync (wrong data). Both are failure modes. Normalize by default; denormalize only with a clear reason.

6. Transactions and ACID

A **transaction** is a group of SQL statements treated as one indivisible unit. You wrap them: `BEGIN;` … your statements … `COMMIT;` to make them permanent, or `ROLLBACK;` to undo everything. **ACID** is the set of four guarantees a good database gives transactions:

- **A — Atomicity** (all-or-nothing): every statement commits, or none do. A crash or rollback midway leaves the database exactly as if the transaction never started.
- **C — Consistency**: the transaction moves the database from one valid state to another, never violating constraints (PK, FK, CHECK, NOT NULL). If any statement would break a rule, the whole transaction is rejected.
- **I — Isolation**: running transactions don't see each other's half-finished, uncommitted work. The result looks as if they ran one after another (how strictly depends on the "isolation level," next).
- **D — Durability**: once `COMMIT` returns "done," the change survives a crash or power loss. This is achieved with a **write-ahead log (WAL)** — the change is written to a log on disk *before* the commit is acknowledged, so it can always be recovered.

Common mistake: Confusing the ACID "C" with the CAP "C." ACID consistency = "constraints stay valid." CAP consistency (a later networking topic) = "all replicas agree on the latest value." They are unrelated concepts that happen to share a letter.

The bank-transfer example — all four letters at once

Transfer $100 from Alice to Bob:

```
BEGIN;
  UPDATE accounts SET balance = balance - 100 WHERE id = 'alice';
  UPDATE accounts SET balance = balance + 100 WHERE id = 'bob';
COMMIT;
```

- **Atomicity** — if the server dies between the two UPDATEs, atomicity rolls back the debit. Money is never taken from Alice without reaching Bob; no $100 vanishes.
- **Consistency** — a `CHECK (balance >= 0)` rejects the whole transfer if Alice doesn't have $100.
- **Isolation** — a "total balance" report running concurrently won't catch the moment $100 has left Alice but not yet arrived at Bob; it sees the system before or after, never in-between.
- **Durability** — once the bank says "transfer complete," a power failure one second later cannot lose it.

Common mistake: Doing multi-step writes (debit + credit, or order + stock decrement) as separate statements *outside* a transaction. A failure between them leaves a broken half-state — money debited but not credited, an order with no stock taken. Wrap related writes in one transaction.

7. Isolation levels and concurrency anomalies

Perfect isolation (every transaction acting as if it's completely alone) is expensive — it requires lots of locking, which slows everyone down. So SQL defines *weaker* levels that trade some isolation for speed. To choose a level you must understand the three classic **read anomalies** — bad things that can happen when transactions overlap:

- **Dirty read** — Transaction 1 reads a row that Transaction 2 has changed but *not yet committed*. If T2 then rolls back, T1 acted on data that never officially existed.
- **Non-repeatable read** — T1 reads a row, T2 updates and commits that *same row*, T1 reads it again and gets a *different value*. The same row changed underneath T1.
- **Phantom read** — T1 runs a range query (`WHERE total > 100`), T2 inserts a new matching row and commits, T1 re-runs the query and now gets *extra rows*. The *set* of rows changed — new "phantoms" appeared.

```
Non-repeatable read timeline
T1                          T2
--------------------        --------------------
read row #7  -> $50
                            UPDATE row #7 = $80
                            COMMIT
read row #7  -> $80   <- same row, different value!
```

The four standard isolation levels, from weakest to strongest, and which anomalies each prevents:

Isolation level	Dirty read	Non-repeatable read	Phantom read
READ UNCOMMITTED	possible	possible	possible
READ COMMITTED	prevented	possible	possible
REPEATABLE READ	prevented	prevented	possible*
SERIALIZABLE	prevented	prevented	prevented

** The bare SQL standard allows phantoms at REPEATABLE READ — but real engines below are stricter.*

How real engines actually behave (frequently tested)

- **PostgreSQL default = READ COMMITTED.** Postgres has *no real* READ UNCOMMITTED — ask for it and you get READ COMMITTED. Its **REPEATABLE READ** uses **snapshot isolation** (each transaction sees a frozen snapshot of the data) and, unlike the bare standard, *also prevents phantom reads*. Its **SERIALIZABLE** uses **SSI** (Serializable Snapshot Isolation): it watches for dangerous read/write dependency cycles and *aborts* one transaction — so your app must be ready to **retry on a serialization failure**.
- **MySQL/InnoDB default = REPEATABLE READ**, and InnoDB's "next-key locking" largely blocks phantoms too.
- The same level name can behave differently across engines — the SQL standard sets *minimums*, not exact behavior. Never assume two databases match just because the level is called the same thing.

The lost update anomaly

There's one more important hazard the standard table doesn't list: the **lost update**. Two transactions both read the same value, both modify it, and both write it back — the second write silently overwrites the first.

```
Both shoppers read stock = 1, both sell the last unit:
T1: read stock=1 -> set stock = 1-1 = 0  (sold!)
T2: read stock=1 -> set stock = 1-1 = 0  (sold AGAIN!)
Result: stock=0 but TWO units sold -> oversold.
```

Common mistake: Believing that simply "using a transaction" or even SERIALIZABLE at READ COMMITTED magically prevents lost updates. At READ COMMITTED it does *not*. You must defend explicitly with one of:

- `SELECT … FOR UPDATE` — locks the row so the second transaction waits until the first commits, then reads the new value.
- **Atomic increment** — `UPDATE products SET stock = stock - 1 WHERE id = 7 AND stock > 0;` does the read-and-write in one indivisible step.
- **Version column** (optimistic concurrency) — add a `version` number; update only if the version hasn't changed, and retry if it has.

Best practice: Most online apps run at READ COMMITTED for throughput and reach for `SELECT … FOR UPDATE` or atomic updates exactly where a race matters (stock, balances, counters). Reserve SERIALIZABLE for the trickiest invariants — and always code the retry loop it needs.

8. Putting it together: orders and payments

An e-commerce checkout uses nearly everything above. The schema: `customers` (1:many) `orders` (1:many) `order_items` (many:1) `products`, plus `payments` (many:1 `orders`).

Placing an order must be ONE transaction that does all of this together:

1. Insert the `order` row.
2. Insert each `order_item`, **snapshotting** the product's name and price at that moment (so a future price edit can't rewrite this invoice).
3. Decrement `products.stock`, protected by `CHECK (stock >= 0)` so you can never oversell.
4. Record the `payment`.

Because it's one transaction, a failure anywhere (card declined, out of stock) rolls the *whole thing* back — no half-created orders, no stock wrongly taken. To stop two shoppers buying the last unit at the same instant, lock the stock row with `SELECT … FOR UPDATE` (or use the atomic `WHERE stock > 0` update). And every money column is `NUMERIC`, never float.

Example — the last unit, done right:

```
BEGIN;
  -- atomic decrement: succeeds only if stock remains
  UPDATE products SET stock = stock - 1
    WHERE id = 7 AND stock > 0;
  -- if 0 rows were affected -> sold out -> ROLLBACK
  INSERT INTO orders (customer_id, total) VALUES (42, 19.99);
  INSERT INTO order_items (order_id, product_id, quantity,
                           unit_price, product_name)
    VALUES (currval, 7, 1, 19.99, 'Blue Mug');  -- snapshot!
  INSERT INTO payments (order_id, amount) VALUES (currval, 19.99);
COMMIT;
```

The atomic decrement plus the `CHECK` constraint make overselling impossible; the snapshot freezes the price; the single transaction guarantees no half-order survives a failure.

Key takeaways:

- A DBMS gives you concurrency control, querying, integrity, crash safety, and indexes that flat files can't — use one for any real data.
- The relational model is tables + rows + typed columns, tied together by primary keys and foreign keys; many:many always needs a join table.
- Always write the `WHERE` on `UPDATE` / `DELETE`; `WHERE` filters rows before grouping, `HAVING` filters groups after; use `LEFT JOIN` to keep unmatched rows.
- Store money in `NUMERIC`, never FLOAT; normalize to 3NF to kill harmful duplication, but snapshot order prices on purpose — that's a real fact, not a violation.
- A transaction (`BEGIN…COMMIT`) gives ACID: atomicity, consistency, isolation, durability (via the WAL). ACID's "C" is about constraints, not CAP's replica consistency.
- Weaker isolation levels allow dirty/non-repeatable/phantom reads; Postgres defaults to READ COMMITTED, MySQL/InnoDB to REPEATABLE READ; lost updates need `FOR UPDATE`, atomic increments, or version columns — a transaction alone won't save you.

Databases II — How Databases Store & Find Data Fast (Indexes, B-Trees, LSM)

In the last section you learned what a database *is*. Now we open the hood and look at the most important question of all: when you ask a database for one row out of a billion, how does it find that row almost instantly? The answer is a small set of clever ideas — **pages**, **indexes**, **B-trees**, and **logs** — that together turn an impossible search into a few quick steps. Understanding them is the difference between an app that feels instant and one that grinds to a halt.

1. The physical reality: data lives in fixed-size pages

A database does **not** read one row at a time from disk. Storage is divided into fixed-size chunks called **pages** (also called **blocks**). A page is the smallest unit of input/output — the smallest amount the database ever reads from or writes to disk. To read one row, the engine reads the *entire page* that contains it into memory. To change one byte, it must eventually write the whole page back.

Real-world sizes: **PostgreSQL** uses 8 KB pages; **MySQL/InnoDB** uses 16 KB pages. (An OS filesystem block is usually 4 KB.) Why pages exist: disks and SSDs (solid-state drives — fast flash storage) are far quicker at moving one big chunk than thousands of tiny scattered reads. A page holds a small header, an array of pointers to rows, and the rows themselves.

> **Key takeaway:** The cost of a query is measured in *pages read*, not rows read. 1,000 rows packed into 20 pages is cheap; the same 1,000 rows scattered across 1,000 pages is 50× more expensive — even though the row count is identical.

2. Why a "full table scan" is slow

A **full table scan** (also called a **sequential scan**) reads *every page* of a table to find matching rows. Imagine a table of 10 million users and the query `WHERE email = 'sam@x.com'` with no index. The engine reads all hundreds of thousands of pages — even though exactly one row matches. The cost grows **O(n)**: "order n", meaning the work grows in direct proportion to the table size. Double the table, double the work.

> **Analogy:** Finding a single word in a 900-page book by reading every page start to finish. That is a full table scan. The back-of-the-book index — a sorted list of terms with page numbers — lets you jump straight there. That index is exactly what a database index does.

But a sequential scan is **not always bad**. If your query returns *most* of the table (say "give me all orders"), reading every page in order is actually the *fastest* way, because sequential reads are disk- and cache-friendly. The database planner deliberately chooses a scan for these **low-selectivity** queries. A

scan is only the villain for **selective** queries — ones that return a tiny fraction of rows — where reading the whole table to find a handful of matches is enormous waste.

> **Common mistake:** Assuming every `Seq Scan` in a query plan is a bug to fix. For a query returning most of the table, the scan is genuinely cheaper than thousands of random index lookups, and the planner is right to pick it.

3. Indexes: the core idea

An **index** is a separate, sorted data structure that lets the engine *jump* straight to matching rows instead of scanning everything. Each index entry stores the indexed column's value plus a **pointer** to where the full row lives. In PostgreSQL that pointer is a `ctid` (a "tuple ID" = page number + slot inside the page). The index is kept sorted by value, so the engine can binary-search it.

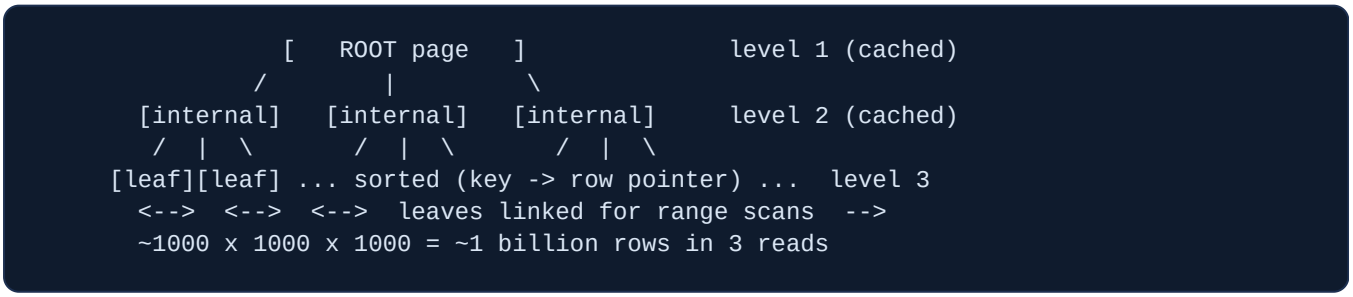
Indexes trade **space and write cost** for **read speed** — a deal that is almost always worth it for the columns you actually search on.

4. B-trees: how O(log n) actually happens

The default index in PostgreSQL, MySQL, SQL Server, Oracle, and SQLite is the **B-tree** — technically a **B+tree**. A B+tree is a tree of pages with these properties:

- **Balanced** — every leaf (bottom node) is the same distance from the top (the root). So every lookup costs the same number of steps, no matter which value you search.
- **High fanout** — each node is one page, and a page holds *hundreds to ~1,000* keys. "Fanout" = how many children each node points to. High fanout means a very wide, very shallow tree.
- **Sorted, linked leaves** — all real keys live in the leaves, and the leaves are linked left-to-right in sorted order.

Here is the magic with concrete numbers. Suppose each node holds about 1,000 keys. Then a tree just **3 levels deep** reaches 1,000 × 1,000 × 1,000 = **1 billion rows**. A lookup walks root → internal node → leaf, doing a quick binary search inside each node. That is only **3 page reads to find one row among a billion** — and the top one or two levels are almost always cached in memory, so often only the final leaf is a real disk read.



That walk takes **O(log n)** time — "log n" grows incredibly slowly. Going from a million rows to a billion adds only one extra level. Compare that to the sequential scan's O(n): from "read every page" to "read

3–4 pages."

Because the leaves are **sorted and linked**, a B-tree also serves **range queries** — `BETWEEN`, `>`, `<`, `ORDER BY`, and prefix searches like `LIKE 'abc%'` — by finding the first matching leaf and walking sideways. This is something a **hash index** (which only does exact "equals" lookups) cannot do.

Key takeaway: A B-tree turns "read every page" into "read 3–4 pages" because each node fans out to ~1,000 children, making the tree shallow. Sorted, linked leaves are what give you fast range scans and `ORDER BY` for free.

5. Clustered vs secondary indexes (a big architectural fork)

There are two ways a database can relate the index to the actual rows, and the two most popular engines pick differently.

- **Clustered index (InnoDB / MySQL):** the *table itself* is stored physically sorted by a key — and that key is the **primary key** (the column that uniquely identifies each row). The leaf nodes *are* the rows. There is exactly one clustered index per table. Looking up by primary key lands you directly on the row — no second hop.
- **Secondary index:** a separate B-tree whose leaves hold the key plus a pointer back to the row. In InnoDB, that pointer is the **primary-key value**. So a search by a secondary column (say email) does *two* tree walks: first the email index gives you a primary key, then the clustered index turns that primary key into the actual row. This is the **"double lookup."**
- **PostgreSQL is different:** it stores the table as an **unordered heap** (a pile of rows in no particular order), and *all* indexes are secondary, pointing at a heap location (`ctid`). There is no clustered index.

Example (InnoDB double lookup): Query `WHERE email = 'sam@x.com'`. (1) Walk the email index → it returns `user_id = 42`. (2) Walk the clustered primary-key index for `42` → that lands on the real row. Two traversals. A covering index (next section) can remove the second one.

Common mistake: Using a random UUID as the primary key in InnoDB without realizing the cost. Because the table is physically sorted by the PK, random keys scatter inserts all over the disk (causing "page splits"), and because every secondary index stores the PK, a fat random key bloats *every* index. Prefer a monotonic (always-increasing) primary key for the clustered index.

6. Composite and covering indexes

A **composite index** covers multiple columns, e.g. on `(last_name, first_name)`. It is sorted by `last_name` first, then by `first_name` within each last name — exactly like a phone book.

Analogy: A phone book sorted by last name then first name lets you instantly find "Smith" or "Smith, John." But it is *useless* for finding every "John" regardless of last name — the sort never groups first names together. This is the **leftmost-prefix rule**.

The **leftmost-prefix rule**: a composite index on `(a, b, c)` serves `WHERE a=…`, `a=… AND b=…`, and all three together — but **not** `WHERE b=…` alone. So **column order matters**. A solid rule of thumb: put **equality** columns first, then `ORDER BY`/sort columns, and put **range** columns (`>`, `<`, `BETWEEN`) last.

A **covering index** contains *every* column a query needs, so the engine answers the query from the index alone and never touches the table — PostgreSQL calls this an **index-only scan**. PostgreSQL and SQL Server support `CREATE INDEX … (email) INCLUDE (name, created_at)`: the `INCLUDE`d columns are stored in the leaf for retrieval but are not sort keys. This eliminates the second hop (the heap or clustered lookup) and is dramatically faster.

Best practice: For a hot read query, make the index *cover* it with `INCLUDE`. The query then never visits the table at all. (PostgreSQL caveat: an index-only scan still checks a small "visibility map" to confirm the row is current — a poorly-vacuumed table can quietly disable it.)

7. The fundamental trade-off: reads vs writes

Indexes speed up reads, but nothing is free:

- **They slow down writes.** Every `INSERT`, `UPDATE`, and `DELETE` must update the table *and every affected index*. Five indexes mean five extra structures to maintain on every write.
- **They cost space.** Each index is a copy of its columns plus pointers. Indexes together can exceed the size of the table itself.
- **They need maintenance.** Over time indexes fragment and bloat, and must be vacuumed or rebuilt (`REINDEX`).

Common mistake: Believing "more indexes = faster database." An unused index is pure overhead — it slows every write and wastes space with zero read benefit. Index for your *real* query patterns, then drop the rest (check `pg_stat_user_indexes` to find unused ones).

8. Write-optimized storage: LSM-trees vs B-trees

A B-tree does **in-place updates**: to change a row it finds the page and rewrites it — a **random write** (writing to a scattered location). Under very heavy write load, random writes become the bottleneck. The **LSM-tree** (Log-Structured Merge-tree) — used by **Cassandra, RocksDB, LevelDB, ScyllaDB, and HBase** — solves this by turning random writes into fast **sequential writes** (appending to the end of a file). Here is the write path:

1. A write is appended to the **WAL** (write-ahead log, for durability — see next section) and inserted into an in-memory sorted structure called the **memtable**.
2. When the memtable fills, it is frozen and **flushed sequentially** to disk as an immutable file: an **SSTable** (Sorted String Table) — key-value pairs sorted by key, plus a small index and a **Bloom filter**.
3. SSTables pile up. A background **compaction** process merge-sorts overlapping SSTables into fewer, larger ones, throwing away overwritten values and deletions (a delete is recorded as a marker called a **tombstone**).

```
write -> WAL (durable) -> MEMTABLE (RAM, sorted)
                              | full -> flush (sequential)
                              v
      SSTable  SSTable  SSTable     <- L0 (immutable, sorted)
                 | compaction (merge-sort, drop dups + tombstones)
                 v
           larger SSTables          <- L1, L2 ... bigger, fewer
```

The catch is **reads**. A key might live in the memtable or in *any* SSTable. To avoid opening every file, each SSTable carries a **Bloom filter** — a tiny probabilistic structure that answers "definitely not in this file" or "maybe in this file." It lets the engine skip files that cannot contain the key (roughly 10 bits per key gives under ~1% false positives).

Aspect	B-tree	LSM-tree
Writes	Random, in-place (slower under heavy load)	Sequential append (very high throughput)
Reads	Predictable, usually 3–4 page reads	May check several SSTables (read amplification)
Extra work	Page splits, fragmentation	Compaction rewrites data (write amplification), latency spikes
Best for	Read-heavy, general-purpose OLTP	Write-heavy, high-ingest, time-series

Common mistake: Believing LSM is universally faster. It wins on write-heavy ingest but adds *read amplification* (checking multiple files) and occasional latency spikes when compaction runs. B-trees give more predictable read latency for everyday transactional apps.

9. The WAL: durability and crash recovery

The **Write-Ahead Log (WAL)** follows one rule: *log the change before you change the data*. Before a transaction commits, its changes are written and **fsync**'d (forced safely to disk) into the sequential WAL. *Only then* is the commit acknowledged to the client. The actual data pages can be updated in memory

and written to disk lazily later, during a **checkpoint** (which flushes dirty pages and lets old WAL be recycled).

> **Example (crash recovery):** Power dies mid-transaction. On restart, the engine **replays the WAL**: it redoes committed changes that hadn't yet reached the data files, and discards uncommitted ones. Result: no lost committed data, no half-written ("torn") pages. The WAL is fast because it is append-only sequential I/O.

PostgreSQL calls this the WAL; InnoDB calls it the **redo log** — same idea. LSM-trees use a WAL too, to protect the in-memory memtable. Tuning knobs (`synchronous_commit`, `innodb_flush_log_at_trx_commit`) trade a little crash-safety for speed.

> **Common mistake:** Treating the WAL as a backup. It exists for crash recovery and durability (replayed on restart), not as a substitute for real backups or for keeping history.

10. The buffer pool: caching pages in RAM

The **buffer pool** (PostgreSQL: `shared_buffers`; InnoDB: the buffer pool, often 50–75% of RAM) is an in-memory cache of recently used pages. Every read and write goes *through* it. When a page is requested, the engine checks the buffer pool first — a **buffer hit** means no disk I/O at all. Modified ("dirty") pages are written to disk later at checkpoints.

This is why the *second* run of a query is often dramatically faster, and why the top levels of a B-tree are effectively free — they stay resident in RAM. Keeping your "working set" (the data you actually touch) in the buffer pool is one of the highest-leverage database optimizations.

11. Query planning and fixing a slow query

SQL is **declarative**: you state *what* you want, and the **query planner** (optimizer) decides *how* — which index, which scan type, which join order. It uses table **statistics** (row counts and value distributions, gathered by the `ANALYZE` command) to estimate **selectivity** (what fraction of rows match) and picks the cheapest plan.

- `EXPLAIN` shows the *estimated* plan without running the query.
- `EXPLAIN ANALYZE` actually runs it and shows *real* timings and row counts. Add `BUFFERS` to see pages hit in cache vs read from disk.

Example (before/after):
Before: `Seq Scan on orders (rows=1) actual time=210ms` — reading the whole table to find one order.
Fix: `CREATE INDEX idx_orders_customer ON orders(customer_id);`
After: `Index Scan using idx_orders_customer ... actual time=0.3ms`, and `BUFFERS` shows shared reads dropping from thousands to a handful. A ~700× speedup from one index.

Reading a plan: look for a `Seq Scan` on a big table feeding a selective filter (smells like a missing index), `Index Scan` / `Index Only Scan` (good), and a big gap between *estimated* and *actual* rows (a sign of stale statistics — run `ANALYZE`). A `Nested Loop` over many rows often means a missing index on the join column.

Best practice — the workflow: (1) find the slow query (`pg_stat_statements` or the slow-query log); (2) run `EXPLAIN ANALYZE` ; (3) spot the `Seq Scan` or expensive filter; (4) add an index on the `WHERE` / `JOIN` / `ORDER BY` columns (equality columns first, range last); (5) consider `INCLUDE` columns to make it covering; (6) re-run `EXPLAIN ANALYZE` and confirm the planner actually *uses* the new index and reads fewer buffers.

Common mistakes that silently disable an index: wrapping the column in a function (`WHERE lower(email)=…`) or casting its type (`WHERE date_col::text=…`) — match the column exactly or build an expression index. A leading wildcard (`LIKE '%abc'`) can't use a normal B-tree (only `'abc%'` can) — use a trigram/full-text index instead. And creating an index does not guarantee it's used: stale stats or low selectivity may make the planner ignore it, so always re-check the plan.

Key takeaways:

- Databases read and write fixed-size **pages** (Postgres 8 KB, InnoDB 16 KB); cost is counted in *pages read*, not rows.
- A **B-tree index** turns an O(n) full scan into an O(log n) lookup — ~3–4 page reads even across a billion rows — and its sorted, linked leaves power range scans and `ORDER BY`.
- **InnoDB** stores rows inside the clustered primary-key index (secondary lookups cost two traversals); **PostgreSQL** keeps an unordered heap with all-secondary indexes.
- **Composite indexes** obey the leftmost-prefix rule (equality columns first, range last); a **covering index** answers a query without touching the table.
- Indexes speed reads but tax every write and consume space — index for real query patterns, not every column.
- **LSM-trees** win on write-heavy workloads (sequential writes + Bloom filters) at the cost of read/write amplification; **B-trees** give predictable reads for general OLTP. The **WAL** guarantees durability and crash recovery, and the **buffer pool** caches hot pages in RAM.
- Diagnose with `EXPLAIN ANALYZE (BUFFERS)`: find the scan, add the right index, and verify the planner actually chose it.

Databases III — Scaling Up: Replication, Partitioning & NoSQL

So far you have used one database on one server. For most apps, that is fine. But as an app grows, one server eventually struggles. This section explains *why* one server isn't enough, and the two main tricks to grow beyond it: **replication** (keep copies of the same data) and **partitioning** (split different data across machines). We'll also meet the **NoSQL** database family and learn about **connection pooling**, a small idea that quietly saves many apps from falling over.

The four ceilings of one server

A "ceiling" here means a limit you eventually hit. A single database server has four separate limits, and the most important lesson is that **each one has a different fix**. Naming them separately helps you choose the right tool.

- **Read load** — too many "read" queries per second. A read is a `SELECT`: fetching data without changing it (showing a product page). Too many reads overwhelm the server's CPU and disk. *(Easiest to fix.)*
- **Write load** — too many "write" queries per second. A write is an `INSERT`, `UPDATE`, or `DELETE`: it changes the data. *(Hardest to fix.)*
- **Storage** — the data grows bigger than one machine's disk can hold (or bigger than its memory can keep fast).
- **Availability** — "available" means up and reachable. One server is a **single point of failure**: if it dies, the whole app is down, and you can't even take it offline for an upgrade without downtime.

> **Key takeaway:** Replication mainly solves *read load* and *availability*. Partitioning (sharding) solves *write load* and *storage*. They fix different problems, so large systems use both together.

Two directions to grow: up vs out

There are two ways to give a database more power.

- **Vertical scaling (scale up)** — buy a *bigger* machine: more CPU, more memory, a faster disk. Simple, no code changes, no new complexity. But there is a hard physical ceiling (the biggest machine that exists), the cost grows faster than the power (a "twice as powerful" box costs much more than twice as much), and it's still a single point of failure.
- **Horizontal scaling (scale out)** — add *more* machines and spread the work. Effectively unlimited, and gives you spare copies for safety. But now machines must talk over a network and stay in agreement, which adds real complexity. Replication and sharding live here.

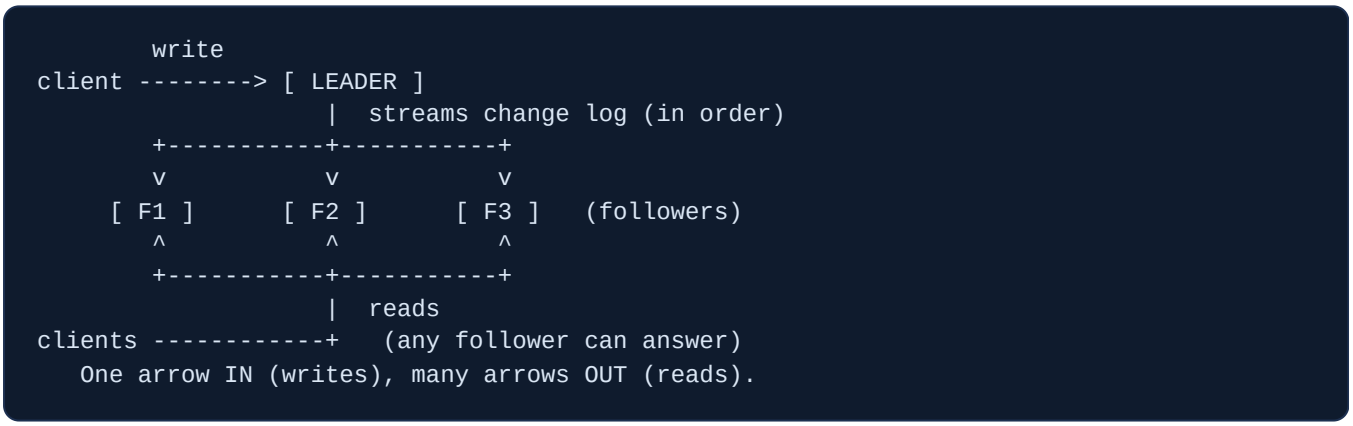
Best practice: Try vertical scaling first — it's underrated. Modern cloud servers reach hundreds of CPU cores and terabytes of memory, which is enough for the vast majority of apps. Reach for horizontal scaling only when a bigger box genuinely can't keep up.

Replication — many copies of the same data

Replication means keeping the *same* dataset on several servers. It gives you more machines to serve reads, a backup ready to take over if one dies, and copies that can sit physically near your users for speed.

The standard shape is **leader/follower** (also called primary/replica; the old term was master/slave). One server is the **leader**. **All writes go to the leader.** The leader keeps a **change log** — an ordered list of every change it made (Postgres calls this the WAL, "Write-Ahead Log"; MySQL calls it the binlog). It streams this log to the **followers**, which replay the exact same changes in the exact same order. Reads can be answered by any server.

Why one leader? Writes must happen in a definite order to keep the data consistent, and the single leader is the one place that decides that order.



Analogy: The leader is the library's master reference copy. Followers are photocopies that visitors read from, so the original isn't mobbed. If the photocopies update a moment after the master, that small delay is **replication lag**.

How fast must followers keep up? Three choices

Mode	Leader replies "OK" when…	Trade-off
Synchronous	at least one follower confirms it saved the write	No data loss, but slower; if that follower is slow or down, writes stall.
Asynchronous (common default)	the leader saves it locally — followers catch up later	Fast and resilient, but if the leader dies before a write reaches a follower, that write is **lost**.
Semi-synchronous	exactly one follower confirms; the rest update async	Practical middle ground: always a second durable copy, with low latency. Common in production.

Replication lag and the "read-your-own-writes" bug

Replication lag is how far a follower trails the leader — usually milliseconds, but it can stretch to seconds under heavy load or network trouble. This causes a classic, confusing bug.

> **Example:** A user changes their display name. The write goes to the *leader*. The page reloads instantly and reads from a *follower* that is 200 milliseconds behind. The follower still has the *old* name, so the user sees the old value and thinks the save failed. The fix is called **read-your-own-writes**: for a few seconds after someone writes, send their reads to the leader (or pin them to one specific replica) so they always see their own latest change.

Read replicas are followers dedicated to answering reads. Adding read replicas is the cheapest, easiest scaling win there is — each one multiplies your read capacity. The catch: your app must *split* its traffic — writes go to the leader, reads go to the replicas.

Failover — when the leader dies

Failover means promoting a follower to become the new leader. Three steps: (1) **detect** the leader is dead (usually a missed heartbeat or timeout); (2) **choose** the most up-to-date follower; (3) **reconfigure** clients and the other followers to point at the new leader.

> **Common mistake:** Treating failover as trivially safe. Real dangers: with async replication, writes that hadn't reached any follower are lost forever. **Split-brain** can occur — two servers both believe they're the leader and accept conflicting writes (the old leader must be "fenced off," i.e. blocked). And a too-twitchy timeout causes needless failovers ("flapping"). Tools like Patroni (Postgres) or managed services (Amazon RDS/Aurora, Google Cloud SQL) handle this carefully so you don't have to hand-roll it.

> **Common mistake:** Thinking replication scales *writes*. It does not — every write still funnels through the single leader. Replication scales reads and adds safety; only **partitioning** scales writes.

Partitioning / sharding — splitting the data itself

Replication copies the *same* data everywhere. **Partitioning** splits *different* data across machines: shard A holds users 1–1,000,000, shard B holds 1,000,001–2,000,000, and so on. ("Partitioning" is the general term; "**sharding**" usually means partitioning across separate servers. A "**shard**" is one of those pieces.) This is the *only* way to scale total writes and total storage past one machine, because each shard has its own leader accepting writes *in parallel*.

You must pick a **partition key** (or **shard key**): the column that decides which shard a row lives on. Choosing it well is the single most important sharding decision.

Two ways to assign rows to shards

- **Range partitioning** — split by ranges of the key (names A–M on shard 1, N–Z on shard 2; or by date). *Good:* efficient ordered scans, like "all orders in March." *Bad:* prone to **hot spots** — if you shard by timestamp, every one of today's writes hits the single "today" shard.
- **Hash partitioning** — run the key through a **hash function** (a function that turns any input into a scrambled fixed-size number) and assign by that. *Good:* spreads load evenly. *Bad:* you lose range scans, because neighboring keys scatter to random shards.

> **Example:** Shard users by first letter (A–M / N–Z). Listing users alphabetically is easy. But a marketing import adds 5 million users whose names start with "A" — one shard melts while the other naps. Hashing the user ID spreads everyone evenly, but now "list all A–M users" must visit every shard.

The hot-spot / hot-key problem

Even hashing can't save you from a single super-popular *key*. In a flash sale, every "add to cart" for one product ID hashes to the *same* shard — that shard drowns while the others idle. (Engineers nickname this the "Justin Bieber problem": one celebrity account overwhelms one machine.) **Mitigations:** add a small random suffix to spread one hot key across several partitions (e.g. `product#0` … `product#9`), then gather them on read; or cache that hot key. Amazon DynamoDB does this automatically — it detects a "hot partition" and splits it.

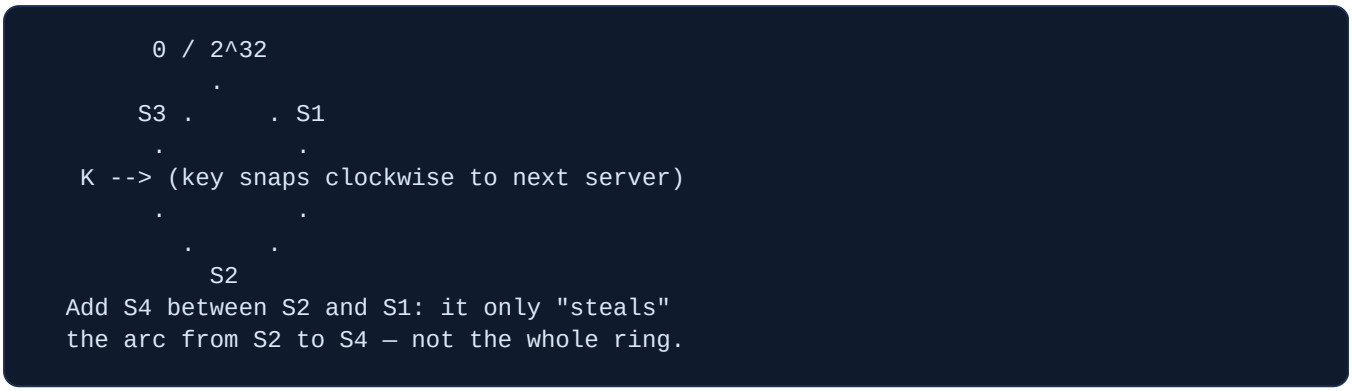
The mod-N trap and consistent hashing

A tempting but broken scheme is `hash(key) % N`, where `N` is the number of servers. It spreads data fine — until you add a server.

> **Common mistake:** Using `hash(key) % N`. With 4 servers you use `% 4`. Add a 5th and switch to `% 5`, and *almost every key* now maps to a different server — forcing a near-total data reshuffle and a cache-miss storm. Use **consistent hashing** instead.

Consistent hashing imagines a ring of numbers (0 up to a huge value, then wrapping back to 0). You hash each *server* onto a point on the ring, and each *key* onto a point too. A key belongs to the first

server you meet going **clockwise**. Now adding or removing a server only reassigns the keys in *one arc* — about **1/N** of the data — instead of all of it.



Real systems improve this with **virtual nodes** (vnodes): each physical server is placed at *many* points on the ring (often ~100–200). This evens out the load and, when a server leaves, spreads its share across many neighbors instead of dumping it all on one. Cassandra, DynamoDB, and many caches use this.

Common mistake: Sharding too early, or on a bad key. **Rebalancing** (moving data when shards are added or removed) happens while the system is live and is never free — queries slow down during the move. And cross-shard work is painful: **joins** (combining rows from two tables) and **transactions** (all-or-nothing groups of writes) that span shards become hard or impossible; totals need a "scatter-gather" across shards. Exhaust vertical scaling, read replicas, and caching first. Pick a *high-variety, evenly spread* shard key, never a low-variety one like country or status.

Why reads are easy and writes are hard

This is the deep idea tying the section together. **Reads are stateless and order-independent**: ask 1,000 replicas the same question and they all give the same answer, so you just add replicas. **Writes must be serialized** — put in a definite order — to keep data consistent, and that ordering needs a single decision point (the leader), which is a chokepoint. Sharding scales writes only by giving up easy cross-shard joins and transactions.

Key takeaway: This is a **CAP-theorem**-flavored trade-off. The more you spread data across machines, the more you drift toward **eventual consistency** (copies agree *eventually*, not instantly) — unless you pay extra latency to keep things strongly consistent. There's no free lunch.

NoSQL — and why it exists

NoSQL means "Not Only SQL." These databases appeared in the 2000s when web-scale companies needed easy horizontal scaling, flexible data shapes, and very high write throughput that single-server relational databases struggled to deliver. The core trade: most NoSQL stores *drop* rich relational

features (joins, fixed schema, multi-row transactions) in exchange for easy scale-out and speed. There are four families.

Family	What it stores	Examples	Good for	Weakness
Key-Value	a giant dictionary: get/put by key	Redis, DynamoDB, Memcached	caching, sessions, rate limiters, leaderboards	can't query by value; no relationships
Document	self-contained JSON-like documents	MongoDB, Couchbase	catalogs, user profiles, evolving data	joins are awkward; you denormalize
Wide-Column	rows by partition key; flexible columns	Cassandra, HBase, Bigtable	time-series, IoT, event logs, write-heavy feeds	must design tables per query up front
Graph	nodes + relationships as first-class	Neo4j	social graphs, recommendations, fraud	doesn't shard easily; weak for bulk analytics

("Schema" = the fixed structure of your data; a "flexible schema" lets different records have different fields. "Denormalize" = store duplicate copies of data together so you don't need a join to read it.)

Example — pick the tool: shopping-cart session → Redis (key-value); product catalog with varying attributes → MongoDB (document); millions of sensor readings per second → Cassandra (wide-column); "people who bought this also bought" → Neo4j (graph); orders + payments + inventory → PostgreSQL (SQL, because you need transactions and joins).

SQL vs NoSQL — a balanced view

SQL databases (PostgreSQL, MySQL) give you a strict schema, strong **ACID transactions** (reliable all-or-nothing changes), powerful joins, and decades of maturity. They're the right default whenever relationships and correctness matter — money, orders, inventory. **NoSQL** gives flexible schemas, built-in horizontal scale, and high throughput, but weaker (often eventual) consistency and limited joins, so you model the data around your exact queries.

Best practice: The line has blurred. Postgres now has **JSONB** (store and query JSON like a document DB) plus partitioning, and "**NewSQL**" systems (CockroachDB, Google Spanner, Vitess) offer SQL *and* horizontal scale. Choose by your **access pattern** and consistency needs, not by hype. Most apps should start with one good relational database.

Connection pooling — a small fix that saves apps

Opening a database connection is surprisingly expensive. PostgreSQL *forks a separate operating-system process* for each connection — several megabytes of memory each — plus a security handshake. It was never built for thousands of simultaneous connections, and `max_connections` (often defaulting to ~100) is a hard ceiling. A busy web app where every request opens its own connection will exhaust the database and crawl.

A **connection pool** keeps a small set of already-open connections and lends them out to requests, taking them back when each request finishes. App-side pools include HikariCP (Java) and pgx (Go); a popular dedicated proxy is **PgBouncer**, which fronts thousands of clients onto a few real backend connections (a PgBouncer client costs ~2 KB versus megabytes for a real connection).

Analogy: A bank counter keeps 10 shared pens for 500 customers, instead of every customer registering their own pen. The pool is the box of pens; PgBouncer is the clerk handing them out and taking them back.

PgBouncer offers **pool modes**: *session* (a real connection is tied to a client for its whole session), *transaction* (the real connection is held only for one transaction — most efficient, but breaks session features like server-side prepared statements and LISTEN/NOTIFY), and *statement* (per query).

Common mistake: Thinking "more connections = faster" and over-sizing the pool. A pool far larger than the database's CPU/disk capacity causes contention and can crash it. Size the pool to the database's *backend capacity* (often a small multiple of CPU cores), not to your request count.

Key takeaways:

- One server hits four separate ceilings — read load, write load, storage, availability — and each needs a different fix.
- **Replication** (same data, many copies) scales *reads* and gives availability; only **sharding** (different data, split across machines) scales *writes* and storage.
- Reads are easy to scale (fan out to replicas); writes are hard because they must be ordered through a single leader — a CAP/eventual-consistency trade-off.
- Use **consistent hashing with virtual nodes**, never `hash(key) % N`; and watch for hot keys that overwhelm one shard.
- **NoSQL** trades joins, fixed schema, and multi-row ACID for scale and flexibility — pick by access pattern, and a relational DB is the right default for most apps.
- Always put a **connection pool** in front of Postgres, and size it to the database's capacity, not your traffic.

Networking I — How Data Travels: IP, TCP/UDP, DNS

Every distributed app is just programs on different machines talking to each other over a network. A **distributed app** means software whose parts run on separate computers and cooperate. When your code calls a database, hits an API, reads from a cache, or asks one microservice to talk to another, that is a *network conversation* — bytes leaving one machine and (hopefully) arriving at another.

Here is the uncomfortable truth that shapes this whole field: the network is the one part you can never make perfectly reliable. Messages get lost, duplicated, reordered, or delayed in unpredictable ways. Almost every hard problem in distributed systems — timeouts, retries, consistency, latency budgets — traces back to how the network actually behaves. So if you understand IP, TCP, UDP, and DNS, you can reason about *why* a service is slow, why a request hangs forever, and what a cryptic error like "connection refused" really means.

Key takeaway: The network is a shared, unreliable medium. Understanding it is not optional trivia — it is the foundation that explains most outages, slowdowns, and "weird intermittent bugs" in distributed systems.

The layered model: solving one problem at a time

Networking is built in **layers**. A layer is a level of responsibility: each one solves a single problem and trusts the layer below it to handle the rest. This keeps the system manageable — the part that encrypts data does not need to know how Wi-Fi works.

The classic teaching diagram is the **OSI 7-layer model** (a textbook reference model with seven levels). But real internet software uses the simpler, practical **TCP/IP 4-layer model**. We will use the 4-layer version because it matches reality.

TCP/IP layer	Its one job	Examples
Application	Speak the app's language	HTTP, DNS, SMTP (email)
Transport	Deliver to the right program	TCP, UDP, QUIC
Internet	Move data between hosts across networks	IP
Link	Move bits across one physical hop	Ethernet, Wi-Fi

Analogy: Think of mailing a letter as a parcel inside a parcel inside a parcel. Your HTTP request is the *letter*. TCP wraps it in an envelope that adds port numbers and reliability. IP wraps *that* in an envelope with the source and destination addresses. The link layer wraps that in one more envelope with the local hardware address for the next hop. Each handler reads only its own envelope and ignores the rest.

This wrapping is called **encapsulation** — each layer adds its own header (a small block of control information) around the data it receives from above. The data being wrapped is the **payload**. The receiving machine unwraps in reverse order, and each layer reads only its own header. This is the single most important mental model in networking.

Packets: why data is chopped up

Data is not sent as one giant blob. It is chopped into small pieces called **packets**. Each packet carries a *header* (addresses, length, and other control fields) plus its *payload* (a slice of your actual data).

Why chop it up? Three reasons. First, many conversations can fairly share one wire instead of one huge transfer hogging it. Second, if one small piece is lost, only that piece is resent — not the whole file. Third, different packets can travel different paths to the destination.

Each network link has a **Maximum Transmission Unit (MTU)** — the largest payload it will carry in one frame. On typical Ethernet this is about **1500 bytes**. Send something bigger and it must be fragmented (split) or it gets dropped.

> **Common mistake:** Ignoring MTU. If you send a payload larger than the path MTU (~1500 bytes), it gets fragmented or silently dropped. This is exactly why large DNS responses sent over UDP fall back to TCP — they no longer fit in one packet.

IP: addressing and routing (best-effort delivery)

IP (Internet Protocol) has one job: get a packet from the source machine to the destination machine across many connected networks. Crucially, IP is **best-effort and connectionless**. "Best-effort" means it tries but makes no promise. "Connectionless" means there is no setup conversation first — each packet is independent.

IP does *not* guarantee delivery, does *not* guarantee order, and does *not* prevent duplicates. This is deliberate: keep the core of the internet dumb and fast, and push reliability out to the edges (to TCP, which we will meet shortly).

IP addresses: IPv4 vs IPv6

- **IPv4** = 32 bits, written as four numbers from 0–255, like `142.250.72.110`. That gives about 4.3 billion addresses — far too few for today's internet. The central pool (IANA) ran out in 2011, and the regional registries exhausted their free pools between 2011 and 2019.
- **IPv6** = 128 bits, written as eight groups of hex digits, like `2001:db8::1` (the `::` collapses one run of zero groups). The address space is astronomically large. As of 2025, global IPv6 adoption sits around **43–48%** (per Google's public measurements) — a slow, decades-long transition, with mobile networks well ahead of corporate ones.

Common mistake: Assuming "everyone is on IPv6 now." Adoption is under half in 2025, so dual-stack setups and NAT'd IPv4 are still everyday reality.

Subnets and CIDR

An IP address splits into a **network part** (which network you are on) and a **host part** (which machine within it). **CIDR notation** writes this as an address plus a slash-number.

Example: `192.168.1.0/24` means "the first 24 bits are the network, the rest is the host." With 32 total bits, 8 bits are left for hosts, giving $2^8 = 256$ addresses: `192.168.1.0` through `192.168.1.255`. A `/24` is a common home-network size.

Grouping machines into **subnets** (sub-networks) lets **routers** — the devices that forward packets between networks — make decisions by network prefix instead of memorizing every individual machine. Routing happens **hop-by-hop**: each router consults its **routing table** ("for this prefix, send the packet out this interface to this next router"). No single router knows the entire path; each only knows the next hop. The `traceroute` tool reveals these hops.

Private addresses and NAT

Some IPv4 ranges are reserved as **private** and never appear on the public internet: `10.0.0.0/8`, `172.16.0.0/12`, and `192.168.0.0/16`. **NAT (Network Address Translation)** lets a whole home or office share one public IP. The router rewrites private source addresses to its single public address and remembers the mapping using port numbers, so replies return to the right device.

Example: A home with 5 devices on `192.168.1.x` all browse through ONE public IP. The router rewrites each outgoing packet's source address and uses port numbers to remember which reply belongs to which device. NAT was a stopgap for IPv4 scarcity and is now everywhere; a side effect is that incoming connections need port-forwarding to reach a device behind it. IPv6's huge space removes the need for NAT.

Ports: finding the right program

An IP address gets you to the right *machine*. A **port** (a number from 0 to 65535) gets you to the right *program* on that machine. The combination **IP:port** is called an **endpoint**.

Analogy: The IP address is the street address of an apartment building. The port number is the specific apartment. You need both to deliver to the right person (the right process).

- **Well-known ports (0–1023):** reserved for standard services — HTTP 80, HTTPS 443, DNS 53, SSH 22.
- **Registered ports (1024–49151):** assigned to specific applications.

- **Ephemeral / dynamic ports (~49152–65535):** the random temporary port your client picks for each outgoing connection.

Transport layer: TCP vs UDP

Both **TCP** and **UDP** add port numbers so the operating system knows which program to hand the data to. The difference is what they *guarantee*.

TCP: reliable, ordered, connection-oriented

TCP (Transmission Control Protocol) gives you a reliable, in-order stream of bytes built on top of unreliable IP. Before any data flows, TCP sets up a connection with the **3-way handshake**:

```
Client                          Server
  | ---------- SYN ----------> |   "here's my start number"
  | <-------- SYN-ACK --------- |   "got it; here's mine"
  | ---------- ACK ----------> |   "got yours"
  |        (data flows)        |
```

SYN means "synchronize" (proposing a starting **sequence number**, a counter for the bytes). ACK means "acknowledge." After these three messages both sides are *ESTABLISHED*. Notice this costs one full **round-trip** (a message there and back) of delay *before* any real data moves. With **TLS** (the encryption layer behind HTTPS) you add another round-trip or two on top.

Once connected, TCP guarantees correctness through three mechanisms working together:

- **Reliability and ordering:** every byte has a sequence number; the receiver sends back ACKs for what it got; anything unacknowledged is *retransmitted* after a timeout. The receiver reassembles bytes in order, so your app always reads a clean, in-order stream.
- **Flow control:** the receiver advertises a *window* — how many more bytes it can accept right now. This stops a fast sender from overwhelming a slow receiver. This is about the two *endpoints*.
- **Congestion control:** TCP also limits its own send rate to avoid overwhelming the *network* itself. The classic scheme is **AIMD (Additive Increase / Multiplicative Decrease)**: grow the send rate slowly while things are fine, then cut it sharply (for example, halve it) the moment a packet is lost, since loss signals congestion. **Slow start** ramps gently at the very beginning.

Common mistake: Confusing flow control with congestion control. Flow control protects a slow *receiver* (the advertised window). Congestion control protects the shared *network* (AIMD/slow start). They are separate mechanisms solving different problems.

Analogy: Flow control is a slow cashier telling a fast bagger "slow down, my counter is full" — about the two people. Congestion control is everyone on a highway easing off the gas when traffic jams — about the shared road. AIMD = ease onto the gas, slam the brakes on a crash.

TCP's key weakness is **head-of-line (HOL) blocking**. Because TCP is one single ordered byte stream, if one packet is lost, every byte after it must wait for that one packet's retransmission — even bytes that already arrived and belong to unrelated requests sharing the connection. One missing packet stalls everything behind it.

Analogy: A single-lane checkout where one stuck customer blocks everyone behind them, even though those shoppers' goods are ready to scan. That stuck customer is the lost packet.

UDP: connectionless, fast, fire-and-forget

UDP (User Datagram Protocol) is the opposite trade. No handshake, no ACKs, no ordering, no retransmission, no flow or congestion control. You send a **datagram** (a self-contained packet) and hope. It may arrive, arrive out of order, arrive twice, or vanish. Its header is tiny (8 bytes). The upside: very low latency, very low overhead, no head-of-line blocking, and support for one-to-many sending (broadcast/multicast).

Aspect	TCP	UDP
Connection	Yes (3-way handshake)	No
Reliable delivery	Yes (ACK + retransmit)	No
In-order	Yes	No
Overhead / latency	Higher	Very low
Best for	Web, APIs, databases, email	Voice, video, games, DNS

Analogy: A phone call is UDP — a dropped half-second is better than replaying it 3 seconds late. Downloading a contract PDF is TCP — every byte must arrive perfectly and in order.

Common mistake: Thinking UDP is "broken" or always worse. It is a deliberate trade. No reliability overhead means lower latency, which is exactly right for live voice, video, and games where a late packet is useless anyway.

QUIC and HTTP/3: the modern twist

QUIC runs over UDP but rebuilds reliability, ordering, and congestion control in software — and crucially gives each logical *stream* its own ordering. So a lost packet stalls only its own stream, not the others, eliminating TCP's transport-level head-of-line blocking. QUIC also folds the connection and TLS handshakes together to cut setup round-trips. **HTTP/3** is simply HTTP running over QUIC. This is the headline reason the web industry moved off plain TCP.

Common mistake: Believing you can dodge head-of-line blocking by "just opening more TCP connections." At the transport level it is inherent to TCP's single ordered stream. The real fix is QUIC/HTTP/3's independent per-stream ordering.

Sockets: the programmer's handle

A **socket** is the operating system's API endpoint your program uses to send and receive over the network. A connection is uniquely identified by a **4-tuple**: (protocol, local IP:port, remote IP:port).

- **Server side:** create socket → `bind` to a port → `listen` → `accept` (returns a fresh socket per client).
- **Client side:** create socket → `connect` → `send` / `recv`.

"Listening on port 8080" means a socket is bound and waiting. Because each connection is a distinct 4-tuple, a single server port can serve thousands of clients at once — each client has a different remote IP:port.

DNS: turning names into addresses

DNS (Domain Name System) is the internet's phone book. Humans use names like `www.example.com`; machines need IP addresses. DNS is a distributed, hierarchical, heavily cached database, read right-to-left: in `www.example.com.` the trailing dot is the implicit *root*, `com` is the **TLD (top-level domain)**, `example` is the domain, and `www` is the host.

A **recursive resolver** does the legwork for you — it asks everyone and returns the final answer (often your ISP's, or public ones like `8.8.8.8` / `1.1.1.1`). **Authoritative servers** hold the real records. The resolution walks the hierarchy, each level *delegating* to the next:

```
"www.example.com?"
      |
      v
Recursive Resolver
  |-> Root server      : ".com lives over there"
  |-> .com TLD server  : "example.com's NS is over there"
  |-> Authoritative    : "A record = 93.184.x.x, TTL 300"
      |
      v
answer cached on the way back
```

Common **record types** to know:

- **A** — hostname → IPv4 address.
- **AAAA** — hostname → IPv6 address.
- **CNAME** — an alias: "this name is really that other name," which must then itself be resolved. It cannot coexist with other records at the same name, and must not sit at the root/apex of a domain.

- **MX** — mail exchange: which server handles email for the domain (with priorities).
- **NS** — which servers are authoritative for a zone. **TXT** — arbitrary text (used for domain verification, email policies).

Every record carries a **TTL (Time To Live)** in seconds — how long it may be cached. A high TTL (e.g. 86400 = 1 day) means fewer lookups and faster responses, but slow to change. A low TTL (e.g. 60–300) updates quickly but creates more query load. DNS queries usually travel over **UDP port 53** (fast, one packet), falling back to TCP for large responses.

Best practice: Before a planned migration, *lower* the TTL days in advance so caches expire by cutover. If you change a record while the TTL is still high, caches keep serving the old value until it expires — this is why people complain a change "won't propagate."

Common mistake: Hardcoding IP addresses instead of names. IPs change; DNS exists precisely so you depend on stable names and let resolution plus TTL handle the rest. Also: assuming CNAME stores an IP (it stores another name), and confusing A (IPv4) with AAAA (IPv6).

Putting it together: what happens when you type a URL

Press Enter on `https://www.example.com/page` and a remarkable amount happens:

1. **Parse the URL:** scheme `https` → port 443, host `www.example.com`, path `/page`.
2. **DNS lookup:** browser checks its cache, then the OS cache, then the recursive resolver, which walks Root → .com TLD → authoritative server and returns the A/AAAA record. The answer is cached at each level per its TTL.
3. **TCP handshake:** open a connection to that IP on port 443 (SYN / SYN-ACK / ACK) — one round-trip.
4. **TLS handshake:** negotiate encryption so the page is private.
5. **HTTP request:** send `GET /page`. It is encapsulated: HTTP bytes → TCP segment (ports + sequence numbers) → IP packet (source/dest IP) → link-layer frame (hardware address for the next hop).
6. **Routing and NAT:** the frame goes to your gateway; routers forward the IP packet hop-by-hop across the internet; NAT may rewrite your source address on the way out.
7. **Server responds:** it unwraps each layer, TCP reassembles the ordered byte stream, the web server reads the request, builds a response, and sends it back the same way.
8. **Browser renders:** bytes reassembled by TCP, decrypted by TLS, parsed as HTML — often triggering more DNS + TCP + HTTP rounds for CSS, JavaScript, and images.

Key takeaway: A single "page load" is dozens of network round-trips, each subject to loss, latency, and setup cost. This is exactly why caching, connection reuse, and HTTP/2 and HTTP/3 multiplexing exist — to shrink that cost.

Key takeaways:

- IP delivers packets to the right machine on a best-effort basis — it may lose, reorder, or duplicate freely; reliability is added above it.
- Ports find the right program; a socket is your program's handle, and a connection is the full 4-tuple (src IP:port + dst IP:port).
- TCP builds a reliable, ordered byte stream on unreliable IP, at the cost of handshake latency and head-of-line blocking; flow control protects the receiver, congestion control protects the network.
- UDP is fast and lossy by design — perfect for voice, video, games, and DNS; QUIC adds modern, per-stream reliability on top of UDP.
- DNS turns names into addresses through a cached hierarchy (Root → TLD → Authoritative); TTL trades freshness against load, so lower it before migrations.

Networking II — The Web Stack: HTTP, TLS, Load Balancing & Caching

In the last networking section we covered the lower-level pipes of the internet: IP addresses, TCP, UDP, and DNS. Those are the roads and the addressing system. This section walks up one floor to the **application layer** — the part developers actually touch every day. We'll learn how the web really works: how a browser asks a server for a page (**HTTP**), how that conversation is kept private (**TLS / HTTPS**), and the infrastructure (load balancers, proxies, CDNs, caches) that makes the web fast, secure, and able to serve millions of people at once.

Analogy: Think of the whole web stack as a busy restaurant. The **CDN/edge** is a food truck on your street with popular dishes pre-made (no trip downtown). The **reverse proxy** is the host at the door who checks your ID and points you the right way. The **load balancer** seats you at whichever kitchen line is shortest. The **kitchens** are the backend servers. And **statelessness** means the kitchen forgets who you are between courses unless you hand back your table number (a cookie).

1. The HTTP request/response model

HTTP (HyperText Transfer Protocol) is a *text-based, request/response* protocol. "Request/response" means a **client** (a browser, a phone app, or a command-line tool like `curl`) sends a request, and the **server** sends back a response. Crucially, it is **client-driven**: the server never speaks first. Nothing happens until the client asks. (That one-way limitation is exactly why WebSockets and Server-Sent Events were later invented — to let servers push data.)

A request has four parts: (1) a **request line** — the method, path, and version; (2) **headers** — key:value metadata about the request; (3) a **blank line**; (4) an optional **body** (form data, JSON, an uploaded file). A response mirrors this with a **status line**, headers, a blank line, and a body.

```
REQUEST                          RESPONSE
GET /products/5 HTTP/1.1         HTTP/1.1 200 OK
Host: shop.com                   Content-Type: application/json
Accept: application/json         Content-Length: 48
<blank line>                     <blank line>
                                 { "id": 5, "name": "Mug" }
```

Methods (also called verbs) tell the server your *intent*:

- `GET` — read/fetch a resource (no body).
- `POST` — create something or submit data.
- `PUT` — replace a whole resource with what you send.
- `PATCH` — partially update a resource.
- `DELETE` — remove a resource.
- `HEAD` — like GET but returns headers only, no body (handy for "did this change?").
- `OPTIONS` — ask what the server allows (used in CORS preflight checks).

Status codes — grouped by their first digit

The server's response always starts with a 3-digit code. The first digit tells you the family:

- **1xx informational** — e.g. `103 Early Hints`.
- **2xx success** — `200 OK`, `201 Created`, `204 No Content`.
- **3xx redirection** — `301` (moved permanently), `302/307` (temporary), `304 Not Modified` (used by caches, explained later).
- **4xx client error** — *you* made a mistake: `400`, `401`, `403`, `404`, `409 Conflict`, `422`, `429 Too Many Requests`.

- **5xx server error** — the *server* failed: `500`, `502`, `503`, `504`.

> **Common mistake:** Confusing `401` with `403`, and `502` with `504`. Memorize these: `401 Unauthorized` = "we don't know *who* you are" (not logged in). `403 Forbidden` = "we know who you are, but you're *not allowed* to do this." For the 5xx pair: `502 Bad Gateway` = a proxy got a *broken* answer from the server behind it; `504 Gateway Timeout` = the proxy waited and got *no* answer in time.

2. Statelessness, cookies, sessions & tokens

HTTP is **stateless**: the server keeps *no memory* of your previous requests. Each request must carry everything needed to handle it. This sounds like a limitation, but it's the secret to **horizontal scaling** — running many identical servers side by side. Because no server "remembers" you, *any* server can handle *any* request, so you can add more servers freely.

But real apps need to remember things like "I'm logged in." The trick: the **client** carries an identifier on every request. The most common way is a **cookie**. The server sends `Set-Cookie: session=abc123`; the browser stores it and automatically attaches `Cookie: session=abc123` to every later request to that site.

Term	What it is	Trade-off
Cookie	A small value the browser stores and auto-sends	Easy, but you must protect it
Session	Server-side data (in Redis/DB) keyed by the cookie's id	Needs shared storage if you have many servers
Token (JWT)	A signed token that *carries* the user's claims itself	No server lookup, but hard to revoke early

Three cookie flags are essential for security: `HttpOnly` (JavaScript can't read it, which blocks theft via XSS attacks), `Secure` (only sent over HTTPS), and `SameSite` (limits cross-site sending, which blocks CSRF attacks). **XSS** = injecting malicious scripts into a page; **CSRF** = tricking your browser into sending a request you didn't mean to.

> **Common mistake:** Storing a session id in a cookie without `HttpOnly`, `Secure`, and `SameSite`. That cookie can then be stolen by a script or ridden by a forged request.

3. HTTP/1.1 vs HTTP/2 vs HTTP/3

HTTP has evolved to be faster. The first big enemy is **head-of-line (HOL) blocking** — when one slow item stuck at the front of a line holds up everything behind it (like one slow shopper jamming the only checkout lane).

HTTP/1.1 (1997): text-based; one connection handles one request at a time. `Connection: keep-alive` lets a connection be reused for several *sequential* requests (avoiding the cost of re-opening it), but they still go one after another. To get parallelism, browsers cheated by opening about **6 connections per host**.

HTTP/2 (2015): same meaning, new wire format. Its wins: (a) **binary framing** (machine-friendly, not text); (b) **multiplexing** — many requests (called *streams*) interleaved over *one* TCP connection, killing the 6-connection hack; (c) **HPACK** header compression, which removes repeated header bytes. *Server Push* existed but is now **deprecated** (Chrome removed it in 2022) — use `103 Early Hints` instead.

HTTP/3 (2022): runs over **QUIC**, a new transport built on **UDP** instead of TCP. QUIC does reliability and ordering *per stream*, so one lost packet only stalls its own stream. It also folds in the TLS handshake (faster setup) and supports **connection migration** — a phone switching Wi-Fi to cellular keeps the same connection via a Connection ID rather than its IP address.

```
HTTP/1.1:  [Req A]--wait--[Req B]--wait--[Req C]   one at a time

HTTP/2:    one TCP pipe:  A1 B1 A2 C1 B2 ...   interleaved
           BUT a dropped packet stalls the WHOLE pipe (TCP)

HTTP/3:    lane A | lane B | lane C   independent
           a drop in lane B stalls ONLY lane B
```

Common mistake: Thinking HTTP/2 fixed *all* head-of-line blocking. It fixed the **application-layer** kind. But because it still rides on TCP, one lost packet stalls every stream — that's **transport-layer** HOL blocking. Only HTTP/3 / QUIC fixes it.

4. REST, APIs & idempotency

REST is a style for designing web APIs. Its rules: **resources** are named by URLs (`/users/42`), you act on them with HTTP **methods**, you transfer **representations** (usually JSON), and it's **stateless**. Use nouns in paths and verbs as methods — write `GET /users/42` , never `GET /getUser?id=42` .

Two properties drive safe API design:

- **Safe** = read-only, changes nothing on the server: `GET` , `HEAD` , `OPTIONS` .
- **Idempotent** = doing it many times has the same effect as doing it once: `GET` , `HEAD` , `OPTIONS` , `PUT` , `DELETE` . **NOT** idempotent: `POST` , `PATCH` .

Why care? Because networks fail. If a request times out, you don't know if it succeeded. You can safely *retry* an idempotent call. Retrying a `POST` , though, might run it twice.

Example: You `POST /charge` a credit card. The response times out. Your code retries — and the customer is charged *twice*. Fix: send an `Idempotency-Key: 7f3a...` header (a unique id for that operation). The server remembers the key and ignores the duplicate. This is the pattern Stripe popularized.

Common mistake: Designing a `GET` with side effects, like `GET /delete?id=5`. GET must be safe — search crawlers and browser prefetchers will happily call it and delete your data. Use the correct verb.

5. TLS / HTTPS — keeping it private and trustworthy

HTTPS is just HTTP carried inside a **TLS**-encrypted channel. TLS (Transport Layer Security) gives three guarantees:

- **Confidentiality** — eavesdroppers see only scrambled *ciphertext*.
- **Integrity** — if anyone tampers with the data, it's detected.
- **Authentication** — you're really talking to the right server, proven by its *certificate*.

A server presents an **X.509 certificate** — a document binding its domain name to a public key, signed by a **Certificate Authority (CA)** that browsers already trust. This forms a **chain of trust** up to a "root" CA stored in your operating system. **Let's Encrypt** made these certificates free and automatic.

```
TLS 1.3 handshake (1 round trip):

Client --ClientHello: ciphers + key share-->        Server
Client <--ServerHello: cert + key share, encrypt now-- Server
        both derive the SAME session key (ECDHE),
        then application data flows. Done in 1 RTT.
```

The key idea: slow *asymmetric* crypto (public/private keys) is used only to *agree on a shared secret key*. Then fast *symmetric* crypto (AES-GCM, ChaCha20) encrypts the actual bulk data. TLS 1.3 finishes the handshake in **one round trip** (TLS 1.2 needed two). It also requires **forward secrecy** (via ephemeral ECDHE keys): even if someone later steals the server's long-term key, they *can't* decrypt traffic they captured in the past. TLS 1.3 also offers **0-RTT resumption** for returning clients (even faster), but it's replay-vulnerable, so use it only for idempotent requests.

Common mistake: Assuming HTTPS makes a request safe to retry. TLS is about *encryption and authentication*, not retry safety. Idempotency is a separate, method-level property.

6. Latency vs throughput vs bandwidth

These three are constantly confused:

- **Latency** — time for *one* trip (delay), measured in milliseconds.
- **Throughput** — actual work done per second (e.g. requests/sec achieved).
- **Bandwidth** — the *maximum capacity* of the pipe (bits/sec).

Analogy: A highway's number of lanes = bandwidth. How long your single car takes end-to-end = latency. Cars arriving per minute = throughput. Adding lanes never makes your drive shorter. Likewise, a cargo ship full of hard drives has *huge* bandwidth but terrible latency (days to arrive); a tiny ping packet has almost no bandwidth but arrives in milliseconds.

Latency has a hard floor: the speed of light is ~1ms per ~100km over fiber, so a cross-continent round trip is naturally ~150ms. A rough order-of-magnitude "latency canon" every engineer should feel:

Operation	Rough time
Main memory read	~100 nanoseconds
Same-datacenter round trip	~0.5 ms
SSD random read	~16–20 microseconds
Cross-continent round trip	~150 ms

Key takeaway: A remote network call is roughly **a million times** slower than reading from memory. So the winning strategy is always: minimize round trips, batch requests together, cache aggressively, and move data physically closer to users.

Common mistake: Trying to fix a slow, round-trip-heavy app by "buying more bandwidth." A fatter pipe doesn't shorten the trip. The fix is fewer round trips, caching, and proximity.

7. Caching everywhere

Caching means storing a copy of a result closer to whoever needs it, so you avoid redoing the work. It exists at every layer of the stack:

```
User
  -> [Browser cache]        governed by Cache-Control: max-age
  -> [CDN edge PoP]         governed by Cache-Control: s-maxage
  -> [Reverse proxy/Varnish]
  -> [App + Redis cache]
  -> [DB buffer pool]
```

HTTP gives you headers to control caching:

- `Cache-Control: max-age=N` — fresh for N seconds.
- `s-maxage` — overrides max-age, but only for *shared* caches like a CDN.
- `no-cache` — may store, but must revalidate before using.
- `no-store` — never store at all (for private/sensitive data).
- `private` — browser only, never a shared cache; `public` — anyone may cache.

When a cached copy might be stale, the cache **revalidates** using a **validator**. An **ETag** is a fingerprint of the content. The cache asks `If-None-Match: "v7"`; if nothing changed, the server replies `304 Not Modified` with *no body* — saving bandwidth. The `stale-while-revalidate` directive serves the old copy instantly while quietly refreshing in the background, hiding the delay from the user.

Best practice: Static files (JS/CSS) → fingerprint the filename (`app.a1b2c3.js`) and cache forever with `max-age=31536000, immutable`; a new deploy is simply a new URL. HTML/API responses → short `max-age` + ETag revalidation. Never cache private, logged-in responses in a shared CDN.

Common mistake: Two opposite errors. (1) Slapping `no-store` everywhere out of fear, destroying performance. (2) Caching a logged-in user's private response in a shared CDN — leaking one user's data to everyone. Cache invalidation is famously hard; stale cache is the #1 cause of "I deployed but users still see the old version." Plan TTLs, ETags, and purge strategy up front.

8. Load balancers

A **load balancer (LB)** spreads incoming traffic across many backend servers. This gives you *scale* (more servers = more capacity) and *high availability* (route around a dead server).

The big distinction is **L4 vs L7** (the OSI layer numbers):

	L4 (transport)	L7 (application)
Routes by	IP + port only	URL path, host, header, cookie
Reads payload?	No (payload-blind)	Yes (content-aware)
Speed	Very fast, low CPU	Smarter, more CPU
Can it terminate TLS?	No	Yes
Example	AWS NLB	AWS ALB, Nginx, Envoy

Balancing algorithms: *Round Robin* (cycle through servers evenly — fine when requests are uniform); *Weighted* (bigger servers get more); *Least Connections* (send to the server with the fewest active requests — best when request durations vary); *IP Hash / Consistent Hashing* (same client always lands on the same server, with minimal reshuffling when servers are added/removed — vital for sharded caches).

Health checks ensure the LB only sends traffic to healthy servers. *Active* checks probe on a schedule ("does `GET /health` return 200?"). *Passive* checks watch real traffic for errors. Use both. **Sticky sessions** pin a client to one server so its session state stays reachable — but this unbalances load and breaks when that server dies.

Common mistake: Using *round robin* when request costs are wildly uneven (some take 50ms, some take 5s) — slow requests pile up unevenly. Use *least connections* instead. And don't lean on sticky sessions as your scaling plan; prefer **stateless servers + shared session storage (Redis)** or tokens, so any server can serve any request.

9. Reverse proxies, CDNs & the edge

A **forward proxy** sits in front of *clients* (e.g. a company's outbound gateway). A **reverse proxy** sits in front of *servers* (Nginx, HAProxy, Envoy). The reverse proxy is your single "front door": it terminates TLS, caches, compresses, routes requests, load-balances, and filters attacks. Clients never touch your app servers directly.

A **CDN** (Content Delivery Network — Cloudflare, Fastly, CloudFront) is a globally distributed network of reverse-proxy caches. Users connect to the nearest **edge PoP** ("point of presence" — a server cluster near them). A cache **hit** is served straight from the edge; a **miss** is fetched from your **origin** (your main server), cached, then served. Benefits: lower latency (content is physically near users), less load on your origin, DDoS absorption, and TLS handled at the edge. "**Edge computing**" pushes not just caching but actual *code* (edge functions) out to those PoPs so even dynamic logic runs close to users.

10. Rate limiting, timeouts, retries & backoff

Rate limiting caps how many requests a client may make in a time window. It protects you from abuse, buggy clients, and overload. Over-limit requests should return `429 Too Many Requests` with a `Retry-After` header telling the client when to try again.

```
TOKEN BUCKET (allows bursts):
  +1 token / 100ms refills the bucket
  10 saved tokens -> a burst of 10 passes instantly,
  then throttles to the refill rate.

LEAKY BUCKET (smooths bursts):
  pour in fast -> drips out exactly 1 / 100ms,
  a steady constant stream no matter the input.
```

Other algorithms: *Fixed Window* (count per clock window — simple, but allows a 2x burst at the boundary), *Sliding Window* (smooths that boundary problem). Token bucket allows controlled bursts and is the most common API choice.

Timeouts are mandatory. Never wait forever — every network call needs a deadline. Without one, a single hung dependency can use up all your threads/connections and take the whole system down.

Retries must be careful. Only retry *idempotent* requests (or those with an Idempotency-Key), and only on *transient* errors (timeouts, `502/503/504`, `429`-with-Retry-After) — never on permanent client errors like `400/401/404`. Use **exponential backoff** (wait 1s, 2s, 4s, 8s…) plus **jitter** (randomize the delay).

```
THUNDERING HERD (no jitter):
  1000 clients fail at t=0, ALL retry at t=1,2,4...
  -> each wave re-crashes the recovering server

WITH JITTER:
  each client picks a random delay in [0, backoff]
  -> retries spread out, the server recovers
```

Common mistake: Retrying with a fixed interval and no jitter. Thousands of clients then retry in perfect lockstep — a **thundering herd** that re-crashes the very service it's waiting on. Always use exponential backoff + jitter, plus a retry cap and a **circuit breaker** (after N failures, stop calling for a cooling-off period).

Best practice: Return the right signal. When you reject for rate limiting, send `429` with `Retry-After` — not a generic `500` or a silent drop. Well-behaved clients then know exactly when to come back.

Key takeaways:

- HTTP is a stateless, client-driven request/response protocol; statelessness is exactly what lets you scale horizontally by adding identical servers.
- Know your status codes cold — `401` (who are you?) vs `403` (you can't do that), and `502` (bad upstream answer) vs `504` (upstream timed out).
- HTTP/2 fixed application-layer head-of-line blocking via multiplexing; HTTP/3 over QUIC (UDP) fixes the remaining transport-layer blocking and adds connection migration.
- TLS uses slow asymmetric crypto only to agree a key, then fast symmetric crypto for the data; TLS 1.3 is a 1-RTT handshake with mandatory forward secrecy.
- Latency, throughput, and bandwidth are different things — a remote call is ~1,000,000× a memory read, so cache at every layer and minimize round trips.
- Only retry idempotent requests on transient errors, with exponential backoff + jitter and a circuit breaker, to avoid double-charges and thundering herds.

Distributed Systems — Many Computers Working as One

A **distributed system** is a group of separate computers that work together over a **network** (the wires and wireless links that carry data between machines) so that, to the person using it, they look and behave like a single system. Each computer is called a **node**. When you open Netflix, search Google, or check your bank balance, you are talking to thousands of nodes pretending to be one.

The computer scientist Leslie Lamport summed up the pain perfectly: *"A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable."* That sentence is the whole chapter in disguise. Let's unpack why we build these systems anyway, and why they are so hard.

Why build a distributed system at all?

Running on one machine is simple. We give up that simplicity for three concrete reasons.

- **Scale** — one machine has a fixed amount of CPU (its "thinking power"), RAM (its short-term memory), and disk (its storage). When your traffic or data grows past the biggest single machine money can buy, you spread the work across many machines. This is called **horizontal scaling**: add more boxes instead of buying one giant box.
- **Reliability / fault tolerance** — one machine is a **single point of failure**: when it dies, everything dies. With copies of your service on many machines, the system survives one machine dying. Oddly, more machines means *more* total failures happen — but the system keeps running through them.
- **Geography / latency** — **latency** means delay. A user in Tokyo and a user in London both want fast replies. Data cannot travel faster than light, so a round trip around the world always takes hundreds of milliseconds. Putting servers near each region cuts that delay.

> **Key takeaway:** We accept the hard problems of a distributed system to get three things one machine can't give: more capacity (scale), survival through hardware failure (reliability), and speed for users far away (geography).

The one new problem that changes everything: partial failure

On a single machine, things either work or the whole thing crashes. Failure is total and obvious. In a distributed system the defining new problem is **partial failure**: some nodes work, some don't — and *you cannot tell which*.

Here is the heart of the difficulty. You send a request to another node and hear nothing back. There are four possible reasons, and they all look **identical** from where you sit:

1. Your request never arrived (it got lost on the network).

2. It arrived and was processed, but the reply got lost coming back.
3. The other node is just slow and is still working on it.
4. The other node is dead.

> **Analogy:** You text a friend and get no reply. Did the text not arrive? Did they read it and not answer yet? Is their phone dead? You honestly cannot tell from your side. That uncertainty *is* partial failure. Every hard idea in this chapter is a strategy for staying correct despite not knowing the answer.

Network partitions and timeouts

A **network partition** is when the network splits so that two groups of healthy nodes can no longer talk to each other. Each group is fine; they just can't reach the other side. Each side may wrongly believe the other side has died.

```
Group A (healthy)         Group B (healthy)
+-----------+    broken    +-----------+
|  A1   A2  |----- X ------|  B1   B2  |
+-----------+    link      +-----------+
     |                          |
"B must be dead"          "A must be dead"
(both are wrong - the LINK died, not the nodes)
```

Your only practical tool for detecting failure is a **timeout**: "if I don't hear back in X seconds, assume it's dead." But a timeout is only a *guess*:

- Too short → you wrongly declare healthy-but-slow nodes dead (a **false positive**) and retry needlessly, sometimes flooding the system.
- Too long → you hang for ages waiting on nodes that are truly dead.

There is no perfect timeout. In fact, deciding "is that node down?" in an **asynchronous network** (one with no guaranteed delivery time) is provably impossible to do perfectly. A famous 1985 result, the **FLP impossibility** (named after Fischer, Lynch, and Paterson), proves that no algorithm can *guarantee* all nodes reach agreement in an asynchronous network if even one node may fail. Real systems get around this with timeouts and a bit of luck — they work in practice, just not with a mathematical guarantee.

> **Common mistake:** Setting timeouts naively. Too short causes "retry storms" that pile load onto an already-struggling system; too long makes everything freeze on dead nodes. Pair timeouts with retries that back off (wait longer each try), add *jitter* (small random delays so clients don't all retry at the same instant), and circuit breakers (stop calling a failing service for a while).

The 8 Fallacies of Distributed Computing

These are false beliefs developers fall into when they treat the network like it's local and perfect. Peter Deutsch (at Sun Microsystems, around 1994) wrote the first seven; James Gosling added the eighth.

#	Fallacy	Reality
1	The network is reliable	Packets drop, cables get cut.
2	Latency is zero	Remote calls are far slower than local ones.
3	Bandwidth is infinite	Big payloads choke the link.
4	The network is secure	Assume it's hostile; encrypt and authenticate.
5	Topology doesn't change	Nodes, routes, and IPs change constantly.
6	There is one administrator	Many teams and configs; coordination is hard.
7	Transport cost is zero	Moving data costs money and CPU.
8	The network is homogeneous	Mixed hardware, protocols, and versions are normal.

Common mistake: Treating a remote call like a normal local function call — ignoring latency, failure, and the cost of turning data into bytes to send (called *serialization*). "Chatty" designs that make hundreds of tiny network calls are one of the top performance killers. Also fallacy 4: don't skip encryption (TLS) and login checks between internal services just because they're "behind the firewall." Assume the network is hostile.

CAP theorem in plain words

Eric Brewer proposed the **CAP theorem** in 2000; Gilbert and Lynch proved it in 2002. It names three properties:

- **C — Consistency** (specifically **linearizability**): every read sees the most recent write. All nodes agree on one current value, as if there were a single copy.
- **A — Availability**: every request to a node that hasn't failed gets a real (non-error) answer, even if that answer might be slightly out of date.
- **P — Partition tolerance**: the system keeps working even when messages between nodes are dropped or delayed.

People say "pick 2 of 3," but here's the honest framing: in any real network, partitions **will** happen. So **P is not optional**. You don't get to "pick CA." The real choice is what to do *during a partition*:

- **CP** — refuse to answer rather than risk giving wrong data.
- **AP** — keep answering with possibly-stale data rather than go down.

Example: A bank ATM picks **CP**: during a partition it's better to say "try again later" than to show a wrong balance and let you withdraw money twice. A social media like-count picks **AP**: a slightly stale count is fine, but the page going down is not.

Common mistake: Believing you can "pick CA." Saying "we chose CA" really means "we didn't think about what happens during a partition." Partitions are unavoidable, so the only real decision is C-vs-A during one.

PACELC — the fuller picture

CAP only describes a partition and ignores the everyday cost of consistency. Daniel Abadi's **PACELC** (2010) fills the gap. Read it as: *if* there's a **P**artition, choose **A**vailability or **C**onsistency; **E**lse (normal operation), choose **L**atency or **C**onsistency.

The insight: even on a perfectly healthy network, keeping copies strongly consistent requires extra coordination round-trips, and those *add delay*. Strong consistency always costs latency.

Class	Meaning	Examples
PA/EL	Stay available in a partition; stay fast normally; weaker consistency	Cassandra, Riak
PC/EC	Strong consistency always, paying latency	Google Spanner, CockroachDB, classic ACID SQL
PA/EC	Available under partition; consistent under normal ops	MongoDB (default)

Consistency models — a spectrum, not a yes/no

Consistency isn't all-or-nothing. It runs from strongest (easiest to reason about, most expensive) to weakest (cheapest and fastest, hardest to reason about).

- **Strong / linearizable** — behaves as if there's one single copy; a read always returns the latest committed write. Simplest to think about, highest coordination cost.
- **Causal** — operations with a cause-and-effect link (like a reply to a comment) appear in that order everywhere. Unrelated operations may be seen in different orders on different nodes. A nice middle ground: it keeps things that "make sense together" in order without forcing global agreement.
- **Eventual** — if writes stop, all copies *eventually* settle on the same value. Until then, reads may be stale or out of order. Cheapest and most available.

Session guarantees make eventual consistency feel sane for one user:

- **Read-your-writes** — after *you* change something, *you* always see your own change (you post a comment and immediately see it).

- **Monotonic reads** — once you've seen a value, you never see an older one (no "going back in time").
- **Monotonic writes** — your writes are applied in the order you made them.

> **Common mistake:** Thinking "eventual consistency" means "consistent within a few milliseconds, basically strong." It only guarantees that copies converge *eventually* if writes stop. Reads can be arbitrarily stale in the meantime. Where users will notice (like seeing their own post), add session guarantees such as read-your-writes.

Replication, quorums, and consensus

Replication means keeping copies of data on several nodes — for durability (you don't lose it if one disk dies), availability, and faster reads. The hard part is keeping the copies *agreeing* while nodes and networks fail.

A **quorum** is a voting trick. With **N** copies, require **W** nodes to confirm a write and **R** nodes to serve a read. If **R + W > N**, the read set and write set must overlap by at least one node — so a read is guaranteed to see the latest write.

```
N = 3 replicas    W = 2 (write reaches 2)   R = 2 (read asks 2)
Since 2 + 2 = 4  >  3, the sets MUST share a node.

 Write went to:  (n1, n2)
 Read asks:           (n2, n3)
                       ^^ overlap => read sees the fresh value
```

A **majority quorum** means more than half: ⌊N/2⌋+1. It tolerates a minority failing and prevents two separate groups from both committing — which would cause **split-brain** (two halves each thinking they're in charge, accepting conflicting writes).

Consensus is the act of making many nodes agree on a single ordered sequence of values — a **replicated log** (an append-only list of operations, in the same order on every node) — despite crashes.

- **Paxos** (Leslie Lamport) — correct and foundational, but famously hard to understand and build.
- **Raft** (Ongaro and Ousterhout, 2014) — designed to be understandable. At any time there's one **leader**; nodes are **leader**, **follower**, or **candidate**. **Leader election** uses randomized timeouts: whoever times out first becomes a candidate, asks for votes, and wins with a majority. **Log replication**: clients send to the leader, which appends the entry and forwards it to followers; once a majority confirms, the entry is *committed*. If the leader dies, a new election runs under a new *term* (a numbered period of leadership).

```
Follower --(no heartbeat, times out)--> Candidate
Candidate --(asks for votes, gets a majority)--> Leader
Leader --(sends heartbeats)--> Followers
Leader crashes --> a Follower times out --> new election (term+1)
```

Raft powers etcd, Consul, CockroachDB, and TiKV; Paxos-family algorithms underpin Google's Chubby/Spanner and ZooKeeper. Consensus needs a majority alive, so it tolerates **⌊(N−1)/2⌋** failures: 3 nodes tolerate 1, 5 nodes tolerate 2.

Best practice: Use an *odd* number of nodes (3, 5, 7). A 4-node cluster gives no more fault tolerance than 3 and makes tie votes (split-brain) possible. Also: don't confuse replication (just copying data) with consensus (agreeing on one order). You can replicate without consensus — but then you have no agreed order, which is exactly how split-brain and conflicting writes happen.

Retries, idempotency, and delivery guarantees

Because the network is unreliable, clients **retry**. But a retry can duplicate work: maybe your first request *did* succeed and only the reply got lost. The fix is **idempotency**: doing the operation twice has the same effect as doing it once.

Example: `SET balance = 100` run twice still leaves 100 — safe to retry. But `ADD 100 to balance` run twice adds 200 — a lost-reply retry just double-charged the customer. The cure is an **idempotency key**: the client attaches a unique ID (say `order-abc123`); the server records which IDs it has processed and ignores duplicates.

Delivery guarantees describe how hard a system tries to deliver a message:

- **At-most-once** — never retry; may lose messages. Simple but lossy.
- **At-least-once** — retry until confirmed; may deliver duplicates. The common, practical default.
- **Exactly-once *delivery*** over an unreliable channel is **impossible**.

Analogy: The **Two Generals Problem**. Two armies on opposite hills must attack at the same time, but every messenger crossing the valley might be captured. Even a confirmation can be lost — and so can the confirmation of the confirmation. Neither general can ever be 100% sure the other got the message. That's why no protocol can guarantee exactly-once delivery.

What real systems actually achieve is **exactly-once *processing*** = at-least-once delivery **+** idempotent processing on the receiving end. When a product says "exactly-once," this combination is what it means — never true network-level exactly-once.

Common mistake: Retrying a non-idempotent operation like "charge card" or "send email." Since a lost reply looks exactly like a lost request, blind retries cause duplicates. Make handlers idempotent (keys, dedup, upserts) *before* you add retries. And don't trust "exactly-once delivery" marketing — you still must write idempotent consumers.

Clocks and ordering — why wall clocks lie

It's tempting to order events across machines by their timestamps. Don't. Each machine's **wall clock** (its real-time-of-day clock) drifts. **NTP** (the protocol that syncs clocks over the internet) can even jump a clock *backward* to correct it. There is no single global "now." Comparing two timestamps from two servers can give the wrong order and silently lose data.

Instead we use **logical clocks** that order by cause-and-effect, not by time:

- **Lamport timestamps** — each node keeps a counter, bumps it on every event, and on receiving a message sets `counter = max(local, received) + 1`. Guarantee: if A caused B, then A's number is smaller. But the reverse isn't true — a smaller number does *not* prove a causal link. Lamport clocks give a total order but lose information about which events were truly simultaneous.
- **Vector clocks** — each node tracks a whole list of counters, one per node. By comparing two vectors you can tell whether two events are causally ordered or genuinely **concurrent** (neither caused the other). This is how systems detect conflicting concurrent writes that must be reconciled. The cost: the vector grows with the number of nodes.

> **Example:** Three people edit a shared document. A Lamport number can say "edit 5 came after edit 3," but can't tell you whether two edits happened at the same time. Vector clocks like `[A:2, B:1, C:0]` versus `[A:1, B:2, C:0]` reveal that neither came before the other — they're concurrent and conflict, so a human or a merge rule must resolve them.

The rare exception is Google **Spanner**'s **TrueTime**, which tames physical clocks using GPS and atomic clocks, then deliberately waits out a small uncertainty window to give globally consistent timestamps.

> **Common mistake:** "Last write wins" by system clock. Clock drift and NTP backward jumps silently corrupt ordering and can throw away newer data. Use logical or vector clocks for causality, not raw wall-clock time.

Distributed transactions: 2PC versus Sagas

A **transaction** is a group of operations that must all succeed or all fail together (its **ACID** guarantee). Inside one database this is easy. Spanning several services or databases — a **distributed transaction** — is hard.

Two-Phase Commit (2PC) uses a **coordinator**:

1. **Phase 1 (prepare/vote)** — the coordinator asks every participant "can you commit?" Each locks its resources and replies yes or no.
2. **Phase 2 (commit/abort)** — if all said yes, tell everyone to commit; otherwise tell everyone to abort.

2PC gives atomicity but is **blocking**: participants hold locks while waiting, and if the coordinator crashes after the vote, participants can be stuck holding those locks indefinitely. The coordinator is a single point of failure. This is a poor fit for microservices because it couples services together and kills availability and throughput.

The **Saga pattern** takes a different path. Instead of one global transaction, run a **sequence of local transactions**, each committing on its own in its own service. If a later step fails, run **compensating transactions** that semantically undo the earlier ones — rather than rolling everything back at once.

Example (e-commerce order): (1) reserve inventory → (2) charge the card → (3) book shipping. If shipping fails, run compensations in reverse: refund the card, release the inventory. Compare with 2PC, where all three would hold locks until the coordinator finally said "commit."

Sagas come in two flavors: **choreography** (each service reacts to the others' events, no central brain) and **orchestration** (one central coordinator drives the steps). The trade-off versus 2PC: no global locks, far more scalable and available — but you give up isolation. Other operations can briefly see half-finished state, and every step needs a real undo action.

Common mistake: Reaching for 2PC across microservices "for safety," then watching it block and strand participants when the coordinator crashes. Prefer Sagas for cross-service workflows. But don't design only the happy path: a Saga has no isolation, so you must write a real compensation for every step and handle the window where intermediate state is visible.

Key takeaways:

- A distributed system makes many computers act as one — for scale, fault tolerance, and low latency — but the network is unreliable and you can't tell "slow" from "dead." That ambiguity (partial failure) is the root of every hard problem.
- Partitions are unavoidable, so CAP's real choice is C-vs-A *during* a partition (CP refuses, AP stays up); PACELC adds that even normally, strong consistency costs latency.
- Consistency is a spectrum (strong → causal → eventual); session guarantees like read-your-writes make eventual consistency feel right to a single user.
- Quorums (R + W > N) and majority consensus (Raft, Paxos) keep replicas agreeing and prevent split-brain; use odd node counts so N=3 tolerates 1 failure, N=5 tolerates 2.
- Networks force retries, so make operations idempotent (idempotency keys); true exactly-once *delivery* is impossible — settle for at-least-once + idempotent processing.
- Don't order events by wall clocks; use logical/vector clocks for causality. For cross-service transactions, prefer Sagas with compensations over blocking 2PC.

Data Engineering — Moving & Shaping Data at Scale

So far in this guide you have learned where data *lives* (databases), how to run software across many machines (distributed systems), and how bytes travel between computers (networking). **Data engineering** sits on top of all three. It is the craft of building the "plumbing" that **moves, cleans, combines, and delivers data** so that dashboards, business analysts, and machine-learning models can use it.

Here is the key difference between a normal software engineer and a data engineer. A software engineer builds the app that *creates* data — every order placed, every button click, every sign-up. A data engineer builds the systems that *collect, reshape, and deliver* that data to the people and programs that make decisions. The product a data engineer ships is not a screen or a feature. It is a **trustworthy, on-time, well-shaped dataset**.

Analogy: Think of a supermarket. The farms (apps) grow the food (data). Data engineering is the entire supply chain in between: the trucks, the cold storage, the washing, the sorting, and the shelves. The shopper (an analyst or an ML model) only sees neat shelves — but none of it works without the supply chain behind the scenes.

Two kinds of work: OLTP vs OLAP

The first thing to understand is that databases serve two very different **workloads** — that is, two different patterns of how data gets read and written.

- **OLTP (Online Transaction Processing)** — the live, operational database behind your app. A "transaction" here means one small unit of business work: "insert this new order", "fetch user 42". OLTP does *many tiny reads and writes*, each touching just a few rows, and it needs to be fast and handle lots of users at once. Examples: PostgreSQL, MySQL.
- **OLAP (Online Analytical Processing)** — the analytics database. It does *a few enormous queries* that scan millions of rows but only look at a handful of columns. Example query: "what was the average order value by region last quarter?" Examples of OLAP systems: Snowflake, BigQuery.

(A **row** is one record — one whole order. A **column** is one field across all records — every order's price.)

Row storage vs columnar storage — and why it matters

How data is physically laid out on disk decides which workload is fast. There are two layouts:

```
ROW-ORIENTED (good for OLTP):
[id1,name1,price1][id2,name2,price2][id3,name3,price3]
  ^-- one whole record sits together --^

COLUMNAR (good for OLAP):
ids:    [id1,   id2,   id3  ]
names:  [name1, name2, name3]
prices: [price1,price2,price3]
  ^-- each column sits together --^
```

Now imagine the query `SUM(price)` over a million orders. On the **row** layout, the computer must read every byte of every record — names, addresses, everything — just to pick out the price. On the **columnar** layout, it reads *only the price block* and ignores the rest.

Columnar storage wins for analytics for three reasons that stack on top of each other:

1. **Less I/O (less data read from disk).** A query touching 3 of 50 columns reads only those 3 — often 90%+ less data.
2. **Better compression.** "Compression" means storing data in fewer bytes. When values of the same type sit together (all countries in one block), patterns repeat and squeeze down tightly — typically 5–10× smaller, sometimes up to ~30× on columns with few distinct values (like "country") using tricks called *dictionary* and *run-length encoding*.

3. **Vectorized execution.** Because same-type values sit in a tidy array, the CPU can apply one instruction to many values at once (a chip feature called SIMD). Aggregations that take minutes on a row store finish in milliseconds.

The trade-off: columnar is **terrible at updating a single row**, because one row is now scattered across many column blocks. That is exactly why OLTP databases stay row-oriented. You can't have it both ways in one system — so you *copy* data from the OLTP system into an OLAP system. That copying is the heart of data engineering.

> **Common mistake:** Running heavy analytics queries directly against your production OLTP database. The storage layout is wrong (row, not columnar) *and* the giant scan steals CPU and memory from live customer transactions, slowing the whole app. Copy the data into an OLAP system and query there.

Getting data across: ETL vs ELT

Both terms describe moving data from a source into an analytics system. They differ only in the order of two steps. **E** = Extract (pull data out), **T** = Transform (clean and reshape), **L** = Load (put it into the warehouse).

- **ETL (Extract → Transform → Load):** clean and reshape the data on a separate machine *first*, then load the finished result. This was standard when storage and compute were expensive.
- **ELT (Extract → Load → Transform):** dump the *raw* data into cheap storage or a powerful cloud warehouse first, then transform it *inside* that warehouse using SQL.

> **Analogy:** ETL is washing and chopping vegetables *before* putting them in the fridge. ELT is stocking the fridge raw and prepping each vegetable only when you cook that recipe. ELT keeps the raw ingredients around, so if you discover a better recipe later, you still have everything you need.

ELT is now dominant. Cloud warehouses have cheap storage and elastic compute (you rent power on demand), so loading raw and transforming later is fast and flexible. Crucially, you **keep the raw data**, so if you find a bug in a transform or need a new metric, you can re-derive it. A popular tool, **dbt**, lets analysts write transforms as version-controlled SQL with built-in tests.

Where the data lives: warehouse vs lake vs lakehouse

Type	What it is	Strength	Weakness
Data warehouse	Structured, cleaned, modeled data. *Schema-on-write* (data must fit a defined shape before loading). Snowflake, BigQuery, Redshift.	Fast, reliable SQL analytics for business reports.	Historically rigid and pricey for messy/unstructured data.
Data lake	Cheap object storage (Amazon S3, Google Cloud Storage) holding *any* file format. *Schema-on-read* (shape is decided when you query, not when you store).	Cheap, flexible, stores images/JSON/logs/anything.	With no rules it becomes a "data swamp" — no transactions, no quality, unreliable to query.
Lakehouse	The lake's cheap storage *plus* warehouse-grade management (transactions, schema rules, time travel) added by a "table format". The modern default.	Cheap and flexible *and* reliable.	More moving parts to understand.

File formats vs table formats — a point everyone confuses

This distinction trips up most beginners, so go slowly.

- **Parquet** and **ORC** are **columnar file formats**. A Parquet file is a single, unchangeable file that stores columns together *and* embeds small statistics (the min and max value in each chunk) so a query engine can skip files it doesn't need. Parquet is the de facto standard.
- **Delta Lake**, **Apache Iceberg**, and **Hudi** are **table formats**. They are *not* a new way to store the data — they store the data *as Parquet files* and add a **metadata / transaction-log layer on top**. That layer records "which files make up this table right now" and gives you ACID transactions (all-or-nothing safe writes), schema evolution (the table's shape can change safely), and time travel (query the table as it looked last Tuesday).

```
LAKEHOUSE LAYER CAKE (read bottom to top)

  Query engines   | Spark, Trino, Snowflake, DuckDB
  ----------------+--------------------------------
  Table format    | Iceberg / Delta: ACID, time
                  | travel, schema, "which files?"
  ----------------+--------------------------------
  File format     | Parquet (columnar files + stats)
  ----------------+--------------------------------
  Object storage  | S3 / GCS (cheap, holds the files)
```

Delta Lake is centered on Spark/Databricks. **Iceberg** is *engine-agnostic* — Spark, Trino, Flink, Snowflake, and DuckDB can all read the same Iceberg table, which is its defining advantage. In 2025 the two converged further (Databricks' Delta "UniForm" can expose a Delta table *as* Iceberg).

> **Common mistake:** Thinking Iceberg or Delta *replace* Parquet. They don't. They store Parquet files and wrap them with a metadata/transaction layer. File format and table format are two different jobs.

Batch vs stream — and "the log" that unifies them

- **Batch processing:** process a finite, bounded chunk of data on a schedule — "all of yesterday's orders, every night at 2am". High throughput, higher latency. Main tool: **Apache Spark**.
- **Stream processing:** process an endless flow of events as they arrive, within seconds or less. Tools: **Apache Kafka** (the pipe that carries events) plus a processor like **Apache Flink**.

There is a deep idea here, made famous by Martin Kleppmann in *Designing Data-Intensive Applications*: **an append-only, ordered log unifies batch and streaming**. A "log" here means a record book you only ever add to the end of, never edit. A *batch is just a bounded slice of a stream*. If every change is an immutable, ordered, **replayable** event in a durable log, then real-time consumers and historical reprocessing can run the *exact same logic* — you just choose where in the log to start reading.

Kafka basics — the log in practice

Kafka is a **distributed, durable, append-only commit log**. Unlike a traditional queue, messages are *not* deleted when read — they are kept so many readers can re-read them.

- **Topic** — a named stream of records, e.g. "orders".
- **Partition** — a topic is split into ordered shards. **Order is guaranteed only *within* a partition**, never across the whole topic. Partitions are the unit of parallelism. Records with the same key (e.g. same customer ID) always land in the same partition, so per-customer order is preserved.
- **Offset** — a record's sequential position within a partition. A consumer's offset is how far it has read. Kafka stores committed offsets in an internal topic called `__consumer_offsets`.
- **Consumer group** — a team of consumers splitting a topic's partitions. **Each partition is read by exactly one consumer in the group.** More partitions → more parallel consumers.

```
Topic "orders"
  Partition 0: [off0][off1][off2] -> Consumer A
  Partition 1: [off0][off1]       -> Consumer B
  Partition 2: [off0]             -> Consumer C
  (add a 4th consumer -> it sits IDLE, no partition left)
```

Analogy: An offset is a bookmark in a book that is never thrown away. The book (the log) keeps all its pages forever. If you crash before saving your bookmark, you simply re-open to the last saved page and re-read — which is where duplicates come from.

Delivery semantics. Kafka's default is **at-least-once**: a consumer processes a record, *then* saves (commits) its offset. If it crashes in between, that record is delivered again and **processed twice**. (At-most-once commits first and risks losing data; exactly-once is possible but harder and costlier.) The practical consequence is huge: **because duplicates can happen, everything downstream must be idempotent.**

Common mistake: Assuming Kafka guarantees global ordering across a whole topic, or that each message is processed exactly once. Order holds only within a partition (use a key to keep related events together), and at-least-once means you *will* see duplicates — design for them.

Reliability disciplines: idempotency, replay, backfills, schema, quality

These disciplines separate a toy script from production data engineering.

Idempotency means: running the same input twice gives the *same* result — no duplicates, no double-counts. The simplest way to get it is to overwrite a target chunk instead of blindly adding to it.

Example: A daily job written as `DELETE FROM orders WHERE day='2026-06-20'; INSERT yesterday's orders;` can be run 5 times and the table is identical every time. Compare a naive `INSERT yesterday's orders` with no delete — run it twice and every order is double-counted. The first is idempotent; the second is a time bomb.

Replayability — because the raw data/log is retained, you can re-run a pipeline from a past point to fix a bug or rebuild a downstream table. **Backfilling** is exactly this: re-populating or correcting *historical* data (you added a column, fixed a transform, or onboarded a new source). A well-built pipeline backfills by re-running one isolated, idempotent period at a time (e.g. dbt's *microbatch* strategy keyed on an `event_time` column), so reprocessing a date range never double-counts.

Schema evolution — sources change shape over time. *Additive* changes (a new optional column) should be absorbed automatically. *Breaking* changes (a renamed, removed, or retyped column) should **halt the pipeline and alert a human** — never silently drop the field, or your dashboard quietly shows wrong numbers.

Data quality testing — validate at every stage: null-rate thresholds, uniqueness/primary-key checks, referential integrity (every order points to a real customer), value-range checks (price > 0), and row-count anomaly detection (today's row count shouldn't be 10× yesterday's). Tools: **dbt tests** and **Great Expectations**.

Common mistake: Building append-only pipelines that aren't idempotent, so any retry or backfill double-counts — and shipping them with no quality tests, so bad data flows silently to dashboards and ML models. Use overwrite-by-partition or upsert-on-key, and add null/uniqueness/range/row-count checks at every stage.

Orchestration: Airflow and DAGs

A real pipeline is many steps that must run in the right order, on a schedule, with retries when something fails. An **orchestrator** coordinates this. The classic tool is **Apache Airflow**, where you describe your pipeline as a **DAG (Directed Acyclic Graph)** in Python. "Directed" = arrows point one way (this step before that). "Acyclic" = no loops, so no step can depend on itself.

```
extract_orders
      |
   +--+--+
   v     v
validate dedupe
   |     |
   +--+--+
      v
 load_warehouse -> build_marts -> refresh_dashboard
 (arrows = dependencies; no cycles allowed)
```

Key takeaway: Airflow *orchestrates* but does not *process* the data itself. It schedules, triggers, and tracks success/failure of jobs — the heavy lifting happens in Spark, Flink, dbt, or the warehouse. (Modern alternatives: Dagster, Prefect, Mage.)

Partitioning data files

In lakes and lakehouses, big tables are physically split into folders by a column — usually a date:

```
s3://bucket/orders/year=2026/month=06/day=20/part-000.parquet
A query "WHERE day='2026-06-20'" reads ONLY that folder
and skips every other day  ->  "partition pruning"
```

This is the file-level cousin of a database index. Best practice: partition on a **low-cardinality column you actually filter on** (a date is ideal — few distinct values, frequently filtered).

Common mistake: Over-partitioning — e.g. partitioning by `user_id` or by the hour on tiny data. You end up with millions of tiny files (the "small-files problem"), which crushes query performance. Keep partitions reasonably large; usually partition by date.

Why this underpins AI and machine learning

Models are only as good as the data feeding them. Data engineering is the supply chain that makes ML possible. **Feature pipelines** (often organized in a *feature store*) compute the inputs a model uses — for instance "average order value over the last 7 days". **Training datasets** are assembled by exactly the batch jobs described here, run over warehouse/lakehouse tables; replayability lets you reproduce the precise dataset a model was trained on.

Common mistake — training/serving skew: Computing a feature one way in nightly batch training and a slightly different way in real-time serving. The model then sees inputs at serving time that don't match what it learned from, and quietly gets worse. Share the same logic via a feature store and the same log, so the two always agree.

Best practice: Don't reach for streaming (Kafka + Flink) just because it sounds modern. Streaming adds real operational cost. If a nightly batch job meets the latency requirement, use it. Choose streaming only when low latency is genuinely needed.

Key takeaways:

- Data engineering builds the reliable pipelines that move, clean, and serve data; it sits on top of databases, distributed systems, and networking.
- OLTP is row-oriented for many small transactions; OLAP is columnar for big analytical scans (less I/O + better compression + vectorized execution). Don't run analytics on your production OLTP database — copy data out.
- ELT (load raw first, transform in the warehouse) beat ETL thanks to cheap cloud storage and tools like dbt. Parquet is a *file* format; Iceberg/Delta are *table* formats that wrap Parquet with ACID, time travel, and schema evolution — the lakehouse.
- An append-only, replayable log (Kafka) unifies batch and streaming. Kafka guarantees order only within a partition and delivers at-least-once, so downstream must be idempotent.
- Idempotency, replay, careful schema evolution, data-quality tests, and partition pruning are what make pipelines safe to retry and backfill. Airflow orchestrates the steps but Spark/Flink/dbt do the work.
- This whole layer is the supply chain for AI/ML — feature stores and shared log logic prevent training/serving skew and make datasets reproducible.

Putting It Together — Designing a Real System & Trade-offs

This is the capstone section. Until now you have learned five separate topics: databases (where data lives), caching (keeping copies of data close by for speed), networking (how machines talk), concurrency (doing many things at once safely), and distribution (spreading work across many machines). In real life you do not use these one at a time. You pull them all together, like levers on a control panel, while designing *one* system.

The good news: you do not need to memorize famous architectures. You need a **repeatable thinking process**. Learn the process and you can reason about any system — even one you have never seen before.

> **Key takeaway:** System design is not about knowing the "right answer." It is about making trade-offs *explicit* and matching them to the requirements. State your assumptions out loud and defend your choices.

The repeatable design process

Follow these seven steps in order, every time:

1. **Clarify functional requirements** — what must the system *do*? (Example: shorten a long web address into a short one, and redirect the short one back.)
2. **Clarify non-functional requirements** — how *well* must it do it? This means scale (how many users), latency (how fast), availability (how often it is up), consistency (how fresh/correct the data must be), and durability (will we lose data).
3. **Back-of-envelope estimate** — rough math for QPS, storage, and bandwidth. (QPS = "queries per second," the number of requests hitting the system each second.) This tells you whether one computer is enough or you need many.
4. **Define the API and data model** — the contract. What does the client send, what comes back, and how is data stored.
5. **Sketch the high-level design** — boxes and arrows: client → load balancer → app servers → cache → database, plus background workers.
6. **Find the bottleneck and scale it** — the bottleneck is the slowest, most overloaded part. Usually it is the database or the hot reads.
7. **Harden for reliability** — redundancy, graceful degradation, idempotency, monitoring.

> **Best practice:** Always state your assumptions and round aggressively. The goal is the right *order of magnitude* (is it 10 servers or 1,000?), not a precise number.

Common mistake: Jumping straight to a complex distributed design before clarifying requirements and estimating scale. This is over-engineering — building for millions of users you do not have. A single PostgreSQL database handles far more than beginners expect.

Back-of-envelope estimation: the cheat sheet

"Back-of-envelope" means quick rough math you could do on the back of an envelope. Memorize these handful of facts and you can estimate almost anything.

- Seconds in a day ≈ **86,400 ≈ 10^5** (one hundred thousand). So **1 million per day ≈ ~12 per second** on average. Flip side: 1 request/second ≈ ~2.5 million/month.
- **Peak ≈ 2–10× the average.** Traffic is bursty (everyone shops at lunchtime). Always state your peak multiplier.
- **Bandwidth** = average payload size in bytes × QPS. (Multiply by 8 if you want bits/second to quote Mbps.)
- **Storage** = records per day × bytes per record × how many days you keep it × replication factor (how many copies you store).
- **Power-of-two sizes:** 2^{10} ≈ 1 KB, 2^{20} ≈ 1 MB, 2^{30} ≈ 1 GB, 2^{40} ≈ 1 TB. One ASCII character = 1 byte; a UUID = 16 bytes.
- **Rough single-node ceilings** (sanity bounds, not gospel): a relational database does ~10,000 writes/second and tens of thousands of reads/second per machine; Redis (an in-memory cache) does 100,000+ operations/second.

Worked example A: a URL shortener (read-heavy)

A URL shortener takes a long web address and gives back a short code (like the ones you see in text messages). Click the short code and it redirects you to the original.

Requirements: create a short code from a long URL; redirect a short code to its long URL. Assume ~100 million new URLs per day, and links must work for years.

Estimation:

- **Writes:** 100,000,000 ÷ 86,400 ≈ **~1,160 writes/second average**. With a peak multiplier of ~10×, call it ~10,000/second at peak.
- **Reads:** a URL shortener has a **read:write ratio around 100:1** (each link is created once but clicked many times). So ~116,000 reads/second average — much higher at peak.
- **Storage:** ~500 bytes per record (long URL + metadata) × 100M/day × 365 days × 5 years ≈ **~90 TB over five years**.
- **Short code length:** 7 characters using Base62 (the 62 URL-safe characters: a–z, A–Z, 0–9) gives 62^7 ≈ **3.5 trillion** possible codes — decades of headroom. We use Base62, not Base64, because Base64 includes `+` and `/`, which have special meaning inside URLs.

Design insight: with 100 reads for every write, **the cache is the system**. Generate a unique ID (from a global counter, or a Snowflake-style 64-bit ID), Base62-encode it as the short code, and store `code → long_url`. The redirect is just a primary-key lookup, so cache it hard: a CDN (a network of edge servers near users) plus Redis in front of the database.

> **Example:** A 301 redirect ("moved permanently") lets the browser cache the destination, so repeat clicks never touch your servers — great for load. But then you lose click analytics. A 302 ("found / temporary") keeps every click hitting you, so you can count it — but you carry all the load. That tension (analytics vs. load) is a real trade-off you should surface, not hide.

Worked example B: e-commerce checkout (write-heavy, money on the line)

Browsing a catalog is read-heavy (cache it). **Checkout is the hard part** because real money, inventory, and outside services are involved. Every topic from earlier shows up here with teeth.

- **Concurrency — the "last item in stock" race:** two buyers click "Buy" for the final unit at the same moment. Without protection, both succeed and you oversell. The fix is a database transaction with row locking (`SELECT ... FOR UPDATE`, which "reserves" the row so the second buyer waits) or an atomic decrement (a single uninterruptible "subtract one"). This is the classic *lost update* problem made concrete.
- **Consistency vs. availability, per piece:** inventory and payment need **strong consistency** — you must never oversell or double-charge. But the product's review count can be **eventually consistent** — it is fine if it is a few seconds stale. Different parts of *one* system sit at different points on the spectrum. That is the key insight.
- **External calls + retries:** the payment gateway might time out *after* it already charged the card. The client retries and — without protection — charges twice. This is exactly why idempotency keys exist (below).
- **Async work:** the confirmation email, invoice PDF, and warehouse notification go on a **queue** (a waiting line of jobs processed by background workers), not on the request path. This keeps checkout fast and survives a downstream outage.

> **Common mistake:** Putting slow or external work (emails, PDFs, third-party calls) on the synchronous request path. This inflates the time the user waits and ties your uptime to someone else's service. Push it to a queue.

Latency numbers every programmer should know

Latency means "how long one thing takes." These rounded numbers (the classic Jeff Dean / Peter Norvig table) drive almost every design decision:

Operation	Rough time
L1 CPU cache reference	~0.5 ns
Main memory (RAM) reference	~100 ns
Read 4 KB randomly from SSD	~150 µs (150,000 ns)
Round trip inside one datacenter	~0.5 ms (500 µs)
Hard-disk (HDD) seek	~10 ms
Round trip California ↔ Netherlands	~150 ms

The mental model: memory is roughly **100× faster** than SSD, which is roughly **100,000× faster** than a cross-continent network round trip. This is *why* we cache in memory and *why* we avoid chatty cross-region calls.

> **Analogy:** Scale these times up so a human can feel them. If reading from L1 cache (0.5 ns) took **1 second**, then reading from RAM (100 ns) would take ~3.5 minutes, reading from SSD (150 µs) would take ~3.5 days, and a cross-continent network round trip (150 ms) would take **~9.5 years**. That is why keeping data close (locality) and caching dominate design.

The five universal trade-offs

This is the heart of the section. Almost every design decision is one of these tugs-of-war.

Trade-off	What pulls each way
Latency vs. throughput	Latency = time for one request. Throughput = requests handled per second. Batching many items together raises throughput but makes each item wait longer. Optimizing one can hurt the other.
Consistency vs. availability (CAP)	When the network "partitions" (machines can't reach each other), you must choose: refuse to serve possibly-wrong data (CP), or stay up and reconcile later (AP). Modern systems are *tunable per operation*. **PACELC** adds: even with no partition, you trade latency vs. consistency.
Cost vs. performance	More replicas, RAM, and regions buy speed and uptime — but cost money. Going from 99.9% to 99.99% uptime can cost roughly 10× the infrastructure.
Simplicity vs. flexibility	A monolith with one Postgres is easy to reason about. Microservices with many data stores scale teams but add operational and consistency complexity. Do not distribute prematurely.
Read-heavy vs. write-heavy	Read-heavy → replicas, caches, denormalize, CDN. Write-heavy → sharding, write-optimized stores (LSM-tree databases), queue-and-batch.

Common mistake: Treating CAP as "pick CP or AP for the whole system." Real systems tune consistency *per operation* (payment = strong, review count = eventual), and PACELC reminds you that you trade latency vs. consistency even when nothing is broken.

Reliability: making it survive failure

Redundancy — no single point of failure

A "single point of failure" is one part whose death takes down everything. Avoid it: run multiple app servers behind a load balancer, keep database replicas with failover, and spread across availability zones or regions. **But redundancy only helps if failover is automatic and tested.** An untested standby gives false confidence — and remember the load balancer, the cache, and the database can each themselves be a single point of failure.

Graceful degradation

Under heavy stress, drop non-essential features instead of falling over completely. Serve a slightly stale cached page, hide recommendations, disable the review widget — but keep checkout working. A worse-but-working experience beats a blank error page.

Best practice: Design for the unhappy path too. Every data view needs a loading state, an empty state, and an error state — and the system needs a plan for "what happens when the payment gateway is down."

Idempotency keys and safe retries

An operation is **idempotent** if doing it twice has the same effect as doing it once. To make "charge the card" idempotent, the *client* generates an **idempotency key** (a unique UUIDv4) per logical request and sends it along. The server stores `key → result`. If a retry arrives with the same key, the server returns the stored result instead of charging again. (Stripe keeps these keys for ~24 hours.)

Retries should use **exponential backoff with jitter**. The formula:

```
delay = min(cap, base × 2^attempt) + random_jitter
```

Example: With base = 200 ms: attempt 1 waits ~200 ms, attempt 2 ~400 ms, attempt 3 ~800 ms — backing off so the struggling server gets breathing room. **Jitter** is a small random amount added to each delay. Without it, thousands of clients that failed at the same instant would all retry at the *exact same moment* and crash the recovering server again — the **thundering herd** problem. Jitter spreads the retries out. Cap the number of attempts (Stripe ≈ 3; others 6–8).

Common mistake: Retrying without an idempotency key (→ duplicate charges/orders), or retrying 4xx client errors. Only retry *transient* failures (timeouts, 429 "too many requests," 503, 408). A 400 or 401 will never succeed on retry — retrying just wastes effort.

Observability: the three pillars

Observability means being able to understand what your system is doing from the outside. It rests on three pillars:

- **Metrics** = numeric time-series (QPS, error rate, p99 latency, CPU). Cheap; perfect for dashboards and alerts. They tell you *what* is wrong.
- **Logs** = timestamped records of events, with rich detail. They tell you *why* it is wrong.
- **Traces** = the journey of one request across all the services it touches, with timing at each hop. They tell you *where* it is slow or failing.

They work together: **metrics alert, traces localize, logs explain. OpenTelemetry (OTel)** is the vendor-neutral standard for emitting all three.

Example (a triage walkthrough): A p99-latency *metric* alert fires. You open a *trace* of a slow request and see the slow hop is the payment service. You read that service's *logs* and find "connection pool exhausted." Three pillars, one root cause.

Common mistake: Looking at *average* latency. The average hides the tail. Watch **percentiles** instead. "p99 = 800 ms" means 99% of requests are faster than 800 ms and the slowest 1% are worse — and that slow 1% is real users having a bad time.

SLI, SLO, SLA, and error budgets

- **SLI** (Service Level Indicator) = the *measured* number, e.g. "% of requests served under 200 ms."
- **SLO** (Service Level Objective) = your internal *target* for that SLI, e.g. 99.9%.
- **SLA** (Service Level Agreement) = a *contractual* promise to customers, with penalties (refunds) if you miss it. It is usually set *looser* than the SLO, so you have a safety margin.
- **Error budget** = 100% − SLO = how much failure you are *allowed*. A 99.9% monthly SLO gives ~43 minutes of budget. Spend it on shipping faster and taking risks; when it is used up, freeze risky changes.

Availability	Downtime per year	Per month
99%	~3.65 days	~7.2 h
99.9% ("three nines")	~8.76 h	~43 min
99.99% ("four nines")	~52 min	~4.3 min
99.999% ("five nines")	~5 min	~26 s

Each extra nine roughly costs 10× more to achieve. Pick the level your business actually needs — most products do not need five nines.

How to reason about any system you meet

This is the takeaway skill. When you face a system you have never seen, ask these questions in order:

1. What does it **do**?
2. What is the **scale** (QPS, storage)?
3. What is the **read:write shape**?
4. Where is the **bottleneck**?
5. What does it **cache**, and what is the invalidation (staleness) story?
6. What is the **consistency requirement** for each piece?
7. What happens when **X fails**?
8. How would I **know** it failed (metrics/alerts)?

If you can answer those eight questions, you can analyze any system. There is rarely one "right" design — only trade-offs made explicit and matched to requirements.

Key takeaways:

- Use the repeatable process every time: requirements → estimate → API/data model → sketch → find the bottleneck → harden for reliability. State assumptions out loud.
- The five universal trade-offs (latency/throughput, consistency/availability, cost/performance, simplicity/flexibility, read-heavy/write-heavy) underlie nearly every decision — and consistency is tunable *per operation*, not for the whole system.
- Memory ≈ 100× faster than SSD ≈ 100,000× faster than a cross-continent round trip — this is why caching and locality dominate design.
- Reliability = redundancy with *tested* automatic failover, graceful degradation, client-generated idempotency keys, and retries with exponential backoff *plus jitter* (jitter prevents the thundering herd).
- Observe with the three pillars (metrics alert, traces localize, logs explain) and measure percentiles like p99, never averages. SLI is measured, SLO is your target, SLA is the contract; error budget = 100% − SLO.
- To analyze any system, ask: scale, read:write shape, bottleneck, caching+invalidation, per-component consistency, failure modes, observability.

Glossary of Terms

The most important terms from across the whole guide, in alphabetical order. Each definition is one plain-English sentence. Use this as a quick reference while you read.

ACID

> Four guarantees (Atomicity, Consistency, Isolation, Durability) that a database transaction either fully happens or fully does not, leaving data valid.

Atomicity

> The "all or nothing" rule: every step in a transaction succeeds together, or none of them take effect.

Buffer pool

> An area of memory where a database keeps recently used data pages so it can avoid slow disk reads.

B-tree

> A balanced tree structure databases use for indexes that keeps data sorted and lets you find a row in a few steps even among millions.

CAP theorem

> The rule that during a network split a distributed system can keep either Consistency or Availability, but not both at once.

CDN (Content Delivery Network)

> A worldwide set of cache servers that store copies of your content close to users so pages load faster.

Columnar storage

> Storing a table by columns instead of rows so analytics queries that read a few columns over many rows go fast.

Concurrency

> Structuring a program so many tasks make progress in overlapping time, whether or not they truly run at the same instant.

Consensus

> A protocol (such as Raft or Paxos) that lets several machines agree on a single value even if some fail.

Consistency (data)

> The property that everyone reading the system sees data that obeys the rules and, ideally, the latest writes.

Deadlock

> A standstill where two or more tasks each hold a resource the other needs, so none can ever proceed.

DNS (Domain Name System)

> The internet's phone book that translates a human name like example.com into a numeric IP address.

Durability

> The guarantee that once a transaction is confirmed, its data survives even a crash or power loss.

ETL / ELT

> Pipelines that move data between systems: ETL transforms before loading, ELT loads raw then transforms inside the warehouse.

Eventual consistency

> A promise that if writes stop, all copies of the data will become identical after a short delay, though they may differ briefly.

HTTP

> The request-and-response language web browsers and servers use to exchange pages and data.

Idempotency

> The property that doing the same operation many times has the same effect as doing it once, which makes retries safe.

Index

> An extra sorted lookup structure on a table that lets the database find rows without scanning everything.

IP (Internet Protocol)

> The addressing and routing scheme that delivers packets of data to the right machine across the internet.

Isolation

> The guarantee that concurrent transactions do not step on each other and behave as if run one after another.

Kafka

> A durable, high-throughput log system for streaming events between producers and consumers in real time.

Latency

> How long one operation takes from start to finish, usually measured in milliseconds.

Load balancer

> A traffic director that spreads incoming requests across many servers so no single one is overwhelmed.

LSM-tree (Log-Structured Merge tree)

> A write-optimized storage design that buffers writes in memory and merges them into sorted files on disk over time.

Mutex

> A lock that lets only one thread at a time touch a shared resource, preventing corruption.

NoSQL

> A family of non-relational databases (key-value, document, column, graph) built for scale and flexible data shapes.

OLAP

> Analytical workloads that scan huge amounts of data to answer reporting and aggregation questions.

OLTP

> Transactional workloads made of many small, fast reads and writes, like placing an order.

Parallelism

> Truly running multiple tasks at the same instant on multiple CPU cores.

Partition (sharding)

> Splitting one large dataset across multiple machines so each holds and serves only a slice.

Process

> A running program with its own private memory, managed by the operating system.

Quorum

> The minimum number of machines that must agree (usually a majority) for a read or write to count as successful.

Race condition

> A bug where the result depends on the unpredictable timing of concurrent tasks.

Replication

> Keeping copies of the same data on multiple machines for safety and faster reads.

Throughput

> How many operations a system completes per unit of time, such as requests per second.

Thread

> A single line of execution inside a process; threads in one process share its memory.

TCP

> A reliable, ordered connection protocol that guarantees every byte arrives correctly, in sequence.

TLS

> The encryption layer that turns plain HTTP into secure HTTPS so data cannot be read or tampered with in transit.

Transaction

> A group of database operations treated as one indivisible unit that fully succeeds or fully rolls back.

UDP

> A fast, connectionless protocol that sends packets without guaranteeing delivery or order.

WAL (Write-Ahead Log)

> A log where a database records a change before applying it, so it can recover after a crash.

Data lake

> A large, cheap store of raw data in many formats, kept for later processing.

Data warehouse

> A structured, query-optimized store of cleaned data built for analytics and reporting.

Frequently Asked Questions

Real questions beginners ask, with short, honest answers. Each links to a section where you can go deeper.

Q: SQL or NoSQL — which should I use?

Start with SQL (a relational database like PostgreSQL) for almost everything. It gives you transactions, joins, and strong guarantees, and it scales further than people think. Reach for NoSQL only when you have a specific need it solves better: enormous scale, very flexible data shapes, or extreme write speed. "It's trendy" is not a reason. See Sections 4 and 6.

Q: Is more threads always faster?

No. Threads help when work waits a lot (network, disk) or when you have many CPU cores for true parallel work. But threads add overhead, contention for locks, and bugs like race conditions. Past the number of CPU cores, extra CPU-bound threads usually make things slower. See Section 3.

Q: Why is my query slow?

Most often a missing index, so the database scans the whole table instead of jumping to the rows it needs. Other common causes: returning too many rows, an N+1 pattern (one query per row in a loop), or a join with no index. Use your database's EXPLAIN command to see the plan. See Section 5.

Q: TCP or UDP?

Use TCP when correctness matters and you cannot lose data (web pages, APIs, file transfers). Use UDP when speed matters more than perfection and a lost packet is fine (live video, voice calls, games). TCP guarantees delivery and order; UDP does not, in exchange for being faster and leaner. See Section 7.

Q: What does "eventual consistency" really mean?

It means copies of your data may disagree for a short time, but if writes stop, they will all converge to the same value. In practice a user might briefly see a slightly stale number (like an old like-count). It is the price of staying fast and available across many machines. See Sections 6 and 9.

Q: What is the difference between latency and throughput?

Latency is how long one thing takes; throughput is how many things you can do per second. A delivery truck has high latency (slow to arrive) but huge throughput (carries a lot). They are different goals and you often trade one for the other. See Section 8.

Q: What is an index, in one breath?

A sorted shortcut. Like the index at the back of a book, it lets the database jump straight to the rows you want instead of reading every page. It speeds up reads but slightly slows writes and uses extra space.

See Section 5.

Q: Why can't a distributed system just have everything — consistent, available, and partition-tolerant?

Because networks fail. When two halves of your system can't talk (a "partition"), you must choose: refuse requests to stay consistent, or answer them and risk serving stale data. That forced choice is the CAP theorem. Partition tolerance is not optional, so the real choice is consistency versus availability during a failure. See Section 9.

Q: What's the difference between a process and a thread?

A process is a running program with its own private memory. A thread is a worker inside that process; all threads in one process share its memory. Sharing makes threads fast to coordinate but dangerous, because they can corrupt the same data. See Section 2.

Q: What is ETL, and is "ELT" just a typo?

ETL means Extract, Transform, Load: clean the data before storing it. ELT swaps the last two: Load raw data first, then Transform it inside a powerful warehouse. ELT has become common because modern warehouses are cheap and fast enough to transform in place. See Section 10.

Q: Batch or streaming?

Batch processes data in big scheduled chunks (e.g. nightly), which is simple and cheap. Streaming processes each event as it arrives, giving near-real-time results at higher complexity. Use batch unless you genuinely need fresh-by-the-second answers. See Section 10.

Q: What is idempotency and why do people keep mentioning it?

An idempotent operation gives the same result whether you do it once or five times. It matters because networks retry: if a "charge card" request is idempotent, a retry won't double-charge. It is the safety net of unreliable networks. See Section 9.

Q: Do I really need this in the AI era?

Yes, more than ever. AI tools generate code fast, but they cannot tell you why your system is slow, why data disagrees across servers, or which trade-off your design should make. Those judgments rest on fundamentals that have lasted 40 years and will outlast today's frameworks. This knowledge makes you the person who can review and steer the AI's output. See Section 1.

Q: What is a WAL and why does my database have one?

A Write-Ahead Log records every change before applying it. If the database crashes mid-write, it replays the log on restart and recovers cleanly. It is how durability (the D in ACID) is actually delivered. See Sections 4 and 5.

Q: How much should I worry about scale on day one?

Usually very little. Most systems never reach the scale where sharding and exotic databases pay off, and premature scaling adds complexity that slows you down. Build something correct and simple first; learn these patterns so you recognize the real moment to apply them. See Section 11.

Revision Cheat Sheet

A fast, dense review of the highest-value facts. Skim before an interview or design discussion.

Latency numbers every engineer should know (orders of magnitude)

Operation	Rough time
CPU cache / register access	~1 ns
Main memory (RAM) access	~100 ns
SSD random read	~100 µs (0.1 ms)
Spinning disk seek	~10 ms
Network round-trip, same datacenter	~0.5 ms
Network round-trip, across continents	~100–150 ms

Memory hook: RAM is ~100× faster than SSD; SSD is ~100× faster than disk; cross-world network is the slowest of all. Avoid the slow layers in hot paths.

ACID (relational guarantees)

- **A**tomicity — all steps or none.
- **C**onsistency — data stays valid by the rules.
- **I**solation — concurrent transactions don't interfere.
- **D**urability — committed data survives crashes (via WAL).

SQL transaction isolation levels (weakest → strongest)

Level	Prevents
Read Uncommitted	(nothing — can read dirty data)
Read Committed	Dirty reads
Repeatable Read	Dirty + non-repeatable reads
Serializable	All anomalies (acts as if one-at-a-time)

Higher isolation = more correctness, less concurrency.

CAP theorem

- During a network **partition**, choose **C** (consistency) or **A** (availability), not both.
- **CP** example: refuse writes to stay correct (many SQL setups).
- **AP** example: keep answering with possibly-stale data (many NoSQL stores).
- Partition tolerance (P) is mandatory in real networks.

TCP vs UDP

	TCP	UDP
Connection	Yes (handshake)	No
Reliable / ordered	Yes	No
Speed / overhead	Slower, heavier	Faster, leaner
Use for	Web, APIs, files	Video, voice, games

SQL vs NoSQL

	SQL (relational)	NoSQL
Schema	Fixed, structured	Flexible
Guarantees	Strong (ACID)	Often eventual
Joins	Yes	Limited / no
Scales by	Mostly bigger machine	Adding machines (shards)
Default choice?	Yes, usually	For specific needs

Index trade-offs & storage engines

- Index = faster reads, **slower writes**, more disk.
- **B-tree**: read-optimized, balanced reads/writes; default in SQL.
- **LSM-tree**: write-optimized (buffers in memory, merges later); used in many NoSQL stores.
- No index → full table scan. Add EXPLAIN to your reflexes.

Deadlock — the four conditions (all must hold)

1. Mutual exclusion (resource held exclusively)
2. Hold and wait (hold one, wait for another)
3. No preemption (can't force release)
4. Circular wait (A waits for B, B waits for A)

Break any one to prevent deadlock (e.g. always lock in the same order → kills circular wait).

Concurrency hazards

- **Race condition**: result depends on timing. Fix with a **mutex**/lock.
- **Deadlock**: everyone stuck. Fix with lock ordering / timeouts.
- Concurrency = overlapping progress; parallelism = truly simultaneous (needs many cores).

OLTP vs OLAP / Warehouse vs Lake

- **OLTP**: many small fast transactions (orders) → row storage.
- **OLAP**: big analytical scans (reports) → columnar storage.
- **Warehouse**: cleaned, structured, query-ready. **Lake**: raw, cheap, any format.

Batch vs Streaming

	Batch	Streaming
Timing	Scheduled chunks	Event-by-event
Freshness	Hours/day old	Seconds
Complexity	Lower	Higher
Tooling	Spark, cron	Kafka, Flink

Back-of-envelope numbers

- 1 day ≈ 86,400 s (~10^5). So 1M events/day ≈ ~12/sec.
- 1 byte char × 1M rows ≈ 1 MB; plan storage from row size × row count.
- Read-heavy? Add caching + read replicas. Write-heavy? Consider sharding / LSM.
- Quorum on N nodes = majority = (N/2)+1; survives losing a minority.

Scaling toolkit (reach in this order)

1. Index & optimize queries
2. Cache (in-memory, then CDN for static content)
3. Read replicas (replication)
4. Bigger machine (vertical scale)
5. Shard / partition (horizontal scale) — last, because it's hardest

Final reminders:

- Pick the simplest design that's correct; add scale only when measured.
- Make network-facing operations **idempotent** so retries are safe.
- Default to SQL + TCP + batch; deviate only with a concrete reason.
- When something's slow, ask "which layer?" before guessing.